

The Algol-24 Programming Language Specification

Status: the conformance pass is complete. Nineteen chapters and eight annexes, 260 rules. Every rule is **decided** — what the language should do — and every rule is claimed by a case: a program in conformance/ , a refusal in refusals/ , or a reproduction in defects/ . None awaits one.

⚠ **This document is now the authority, and the implementation is measured against it** [1.1]. Where the two disagree, **31 defects** in Annex F say so and carry a reproduction that passes while the fault persists. A rule ahead of the implementation says which of three things it is — NOT YET IMPLEMENTED , PARTLY IMPLEMENTED , or PLANNED — a later generation — and spec/spec.sh enforces that each points somewhere.

Fifteen divergences between the interpreter and the C back end are recorded in Annex C, six of them silent. Annex D's eighteen advisory notes are all resolved. Annex H holds nine changes planned for later generations.

1. Introduction

Algol-24 is a Pascal-flavoured, gradually typed language. This document specifies its lexis, syntax, and semantics.

1.1 Authority

⚠ **This specification is the authority.** Where an implementation differs from a rule, the implementation is in error, and the difference is recorded as a numbered defect in Annex F with a program that reproduces it.

That was not always so. Every rule here began as a description of what the tree-walking interpreter in compiler/*.a24 does, verified by **running** it rather than by reading it — and most rules still are exactly that. A rule is only allowed to depart from the implementation by an explicit decision, and when it does it carries a marker:

⚠ **NOT YET IMPLEMENTED.** ... See DEF-*nn*.

⚠ **A rule without such a marker describes behaviour that was observed.** A rule with one describes behaviour that was decided. The distinction is what keeps the document trustworthy, and it is why the markers are in the normative text rather than in an annex.

⚠️ **The specification does not hedge.** Where a behaviour is kept despite looking wrong, it is stated flatly and the misgiving goes to Annex D. A specification that argues with itself cannot be conformed to.

1.2 Conformance

An implementation conforms if, for every program, it produces the output and the exit status this document requires. Diagnostics are part of that surface: their wording is specified, because it is what a user sees and what one implementation can be compared against another by.

Where a rule is marked **compile-only** or **interpret-only**, it constrains that processor alone.

1.3 How to read a rule

Every normative statement carries an immutable identifier and its evidence:

[SRC-000] An example rule, stating one thing that a single test can prove or disprove.

```
interpreter  compiler/Scanner.a24  ScanToken
compiler     bootstrap/algol.c     alg_error
unit         Scan A Whole Program
conformance  TBD
```

⚠️ **Identifiers are permanent.** They are assigned once, never reused and never renumbered; sections may be renamed and reordered freely. Numbering the sections instead would mean that inserting one clause silently rots every citation to everything after it.

The trailer is machine-readable and `spec/spec.sh` checks every line of it. The four keys do different jobs, and two of them are easy to confuse:

Key	Names	Answers
interpreter	a file and symbol in compiler/*.a24	where the authority implements this
compiler	a file and symbol in bootstrap/algol.c	where the C back end implements it, when it has a say
unit	a test in the compiler/ suite	evidence the rule <i>transcribes the authority accurately</i>
conformance	a program in conformance/	a check that <i>any implementation</i> obeys the rule

⚠️ **unit is not conformance evidence, and the distinction is not pedantic.** A unit test reaches into `algolc`'s own classes — it constructs a `Scanner` and asserts token types — so it tests the implementation, not the language. It is still worth citing, because [1.1] makes the interpreter normative and a test pinning the

interpreter's behaviour therefore pins the language's. But another implementation has no `Scanner` class to test, so a unit test can never be run against it. Only a conformance program can.

Every rule is claimed by a case, cited as `conformance`, `refusal` or `defect`. Which of the three is decided by one question — is the interpreter right? — and `conformance/README.md` explains it. Where no case existed the value was the literal `TBD`, so the gap was stated rather than left to be discovered; **that backlog is now empty**, and `spec.sh` still requires the line so a new rule cannot be added without one.

⚠ A rule may cite more than one, and most cases pin more than one rule. A partly implemented rule cites a conformance program for the half that works and a defect for the half that does not, so the per-kind counts overlap and do not sum to the total. `spec.sh --coverage` reports them separately.

`unit` is cited where a test happens to pin the rule and omitted where none does. It is deliberately not mandatory: a unit test is a test of `alg`, and there is no intention that every rule of the language should have one. The coverage report names the rules that lack one.

Non-normative material — annexes, notes, and anything under a heading marked (*non-normative*) — carries no identifiers and constrains nothing.

2. Notation

The syntax is given in Extended Backus-Naur Form, following the Go specification's conventions:

```
Production = production_name "=" [ Expression ] "." .
Expression = Term { "|" Term } .
Term       = Factor { Factor } .
Factor     = production_name | token [ "..." token ] | Group | Option | Repetition .
Group      = "(" Expression ")" .
Option     = "[" Expression "]" .
Repetition = "{" Expression "}" .
```

Lowercase production names denote lexical tokens; uppercase names denote non-terminals. Tokens are enclosed in double quotes.

The form `a ... b` means the set of characters from `a` through `b` inclusive. The symbol `∅` marks an empty alternative.

⚠ A quoted token in this document is matched **case-insensitively if it is a keyword** and exactly otherwise. See [SRC-010].

3. Source code representation

3.1 Characters

[SRC-001] Source text is **UTF-8**. A character, not a byte, is the unit of measurement and of subscripting.

⚠ **NOT YET IMPLEMENTED.** The implementation treats source and strings as bytes and attaches no encoding to either. See DEF-01.

```
interpreter  compiler/Scanner.a24  ScanTokens
compiler     bootstrap/algol.c     alg_length
unit         Scan A Whole Program
defect       DEF-01-text-is-bytes.a24
```

[SRC-002] Outside comments, string literals and character literals, every character must be one the scanner recognises — a letter [SRC-005], a digit, or an operator or item of punctuation [LEX-012]. Any other is an error reading [line N] Error: Unexpected character: C .

⚠ **PARTLY IMPLEMENTED.** The implementation refuses every non-ASCII byte here, including letters that [SRC-005] admits. See DEF-01.

```
interpreter  compiler/Scanner.a24  ScanToken
unit         Scan Unrecognized Character Is Recorded
refusal      0001-unexpected-character.a24
```

[SRC-003] Inside a comment, a string literal or a character literal, any byte is permitted and is carried through unchanged. A program may therefore hold text in any encoding, and the language will neither interpret nor validate it.

```
interpreter  compiler/Scanner.a24  ScanString
compiler     bootstrap/algol.c     alg_string
conformance  0001-source-is-utf8-text.a24
```

[SRC-004] Length of a String is its count of **characters**, and subscripting a String yields the character at that position. Length('café') is 4, and 'café'[3] is é .

The same holds for every operation that counts or indexes text — Copy , Pos , and Ord [16.2].

⚠ **NOT YET IMPLEMENTED.** The implementation counts bytes: Length('café') is 5 and 'café'[3] is the first byte of a two-byte sequence. See DEF-01.

```
interpreter  compiler/Interpreter.a24  LengthNative
compiler     bootstrap/algol.c         alg_length
defect       DEF-01-text-is-bytes.a24
```

Length('café') is 5, not 4, in both processors. Verified.

⚠️ **[SRC-002] and [SRC-003] together mean an identifier is ASCII while a string is not.** The restriction is on the *program text the scanner reads*, not on the data a program may carry.

3.2 Letters and digits

[SRC-005] The following classes are used by the grammar:

```
letter          = "a" ... "z" | "A" ... "Z" | "_" | unicode_letter .
decimal_digit   = "0" ... "9" .
identifier_mark = "?" | "!" .
```

`unicode_letter` is any character Unicode classifies as a letter.

⚠️ **PARTLY IMPLEMENTED.** The implementation admits only the ASCII forms. See DEF-01.

```
interpreter  compiler/Scanner.a24  IsAlpha
unit         Scan Identifier With A Question Mark
conformance  0002-letters-and-digits.a24
```

⚠️ **An identifier mark is not a letter.** `?` and `!` may appear *within* an identifier but may not begin one [LEX-007], so `Gate?` and `Gate!` are each a single word — one word to the scanner, and one word to double-click — while `?` and `!` alone are not identifiers at all. `_` is a letter and may lead.

Neither mark is an operator. The language spells negation `not` and inequality `<>`, so nothing else wants the characters.

3.3 Line termination

[SRC-006] A line ends at `#10`. A `#13` immediately preceding it is part of the terminator and is absorbed with it, so a file with CRLF endings and the same file with LF endings report identical line numbers.

⚠️ The two cases below are a **pair**, and neither proves the rule alone: they are the same program with different line endings, and the rule is that they report identically.

```
interpreter  compiler/Scanner.a24  ScanToken
unit         Scan Newline
unit         Scan Comment Ends At Newline
conformance  0006-line-endings-lf.a24
conformance  0006-line-endings-crlf.a24
```

[SRC-007] Any **other** `#13` is not a terminator. In source it is whitespace [SRC-008]; in data it is ordinary text that `ReadLn` returns [RT-016].

⚠ A file whose only line endings are `#13` therefore holds **one line**, in source and in data alike. Every diagnostic in such a source file reports `line 1`.

⚠ A sharper consequence, from this rule meeting [LEX-001]: a comment runs to `#10` or to end of file, so a `// comment in a file with no #10 anywhere swallows the rest of the file`. A CR-only source beginning with a comment is therefore an empty program that runs and does nothing, rather than one that fails to compile.

This follows from having a single rule rather than three, and it is the reason for having one: `#10` terminates everywhere, `#13#10` counts once, and a stray `#13` is never a terminator in either subsystem. Admitting a lone `#13` as a third convention would buy compatibility with a format no live system produces, at the cost of changing how `ReadLn` splits data — so a program reading text with embedded `#13` bytes that are not line endings would start dividing it differently.

```
interpreter  compiler/Scanner.a24  ScanToken
conformance 0007-carriage-return-only.a24
conformance 0009-comment-swallows-a-cr-only-file.a24
```

A file with CRLF endings and the same file with LF endings report identical line numbers. A lone `#13` between two statements separates them as any other whitespace would. Verified.

[SRC-008] Whitespace is the space, `#9` and `#13`. It separates tokens and is otherwise insignificant.

```
interpreter  compiler/Scanner.a24  ScanToken
conformance 0003-line-termination.a24
```

[SRC-009] The final line of a file need not be terminated.

```
interpreter  compiler/Scanner.a24  IsAtEnd
conformance 0004-final-line-need-not-be-terminated.a24
```

3.4 Case

[SRC-010] Keywords are matched case-insensitively. `begin`, `Begin` and `BEGIN` are the same keyword.

```
interpreter  compiler/Scanner.a24  ScanIdentifier
unit         Scan Keywords
conformance 0005-keywords-are-case-insensitive.a24
```

[SRC-011] Identifiers are matched **case-insensitively**, as keywords are. `Xyz` and `xyz` are one name, and declaring both in one scope is a duplicate [VAR-007].

⚠ Folding is **ASCII-only**. `Straße` and `STRASSE` are different names, because the language carries no Unicode case tables and full folding is a far larger commitment than admitting Unicode letters.

⚠ Only the *lookup* is folded. A diagnostic quotes the lexeme **as written**, so a program declaring `Xyz` and misspelling it `xyZ` is told about `xyZ`.

⚠ **NOT YET IMPLEMENTED.** The implementation matches identifiers exactly, so `Xyz` and `xyz` are two names. See DEF-02.

```
interpreter  compiler/Scanner.a24  ScanIdentifier
unit        Scan Identifier
defect      DEF-02-identifiers-are-case-sensitive.a24
```

⚠ The asymmetry is deliberate and is the one place the language departs from Pascal's uniform case-insensitivity. A program may declare `Count` and `count` as two variables; it may not declare a variable named `Begin`.

4. Lexical elements

4.1 Comments

[LEX-001] A comment begins with `//` and runs to the end of the line, or to the end of the file if no `#10` follows. It is discarded and forms no token.

```
interpreter  compiler/Scanner.a24  ScanToken
unit        Scan Comment
unit        Scan Comment Ends At Newline
conformance  0011-comments.a24
```

[LEX-002] `///` is not a distinct form. The scanner sees `//` followed by a comment whose first character is `/`, and treats it as any other comment.

```
interpreter  compiler/Scanner.a24  ScanToken
conformance  0011-comments.a24
```

The project writes documentation comments as `///` by convention, and tools may treat them specially. The language does not.

[LEX-003] There are no block comments and no nesting. `{ ... }` and `(* ... *)` are not comments. The braces are refused as unexpected characters [SRC-002]; the parenthesis form scans as a `(` followed by a `*`, which is not a prefix operator, and fails with `Expect expression!`.

```
interpreter  compiler/Scanner.a24  ScanToken
refusal     0002-block-comment-braces.a24
```

4.2 Tokens

[LEX-004] A token is an identifier, a keyword, a literal, or an operator or item of punctuation. Whitespace and comments separate tokens and are otherwise discarded.

```
interpreter  compiler/Scanner.a24  ScanTokens
unit         Scan Tokens
conformance  0017-tokens-and-separation.a24
```

[LEX-005] Where a shorter and a longer token both match, the longer is taken. `<` followed by `>` is one `<>`; `<` followed by anything else is a `<` on its own.

```
interpreter  compiler/Scanner.a24  ScanToken
unit         Scan Less Is Not Greedy
conformance  0012-operators.a24
```

`<<><=<` scans as `<`, `<>`, `<=`, `<` — four tokens.

[LEX-006] There is no automatic semicolon insertion. A line ending is whitespace and never stands in for a `;`.

```
interpreter  compiler/Scanner.a24  ScanToken
refusal     0004-no-semicolon-insertion.a24
```

4.3 Identifiers

[LEX-007] An identifier is a **letter** followed by any number of letters, digits and identifier marks. `letter`, `decimal_digit` and `identifier_mark` are as defined in [SRC-005].

```
identifier = letter { letter | decimal_digit | identifier_mark } .
```

`Gate?`, `Send!`, `_under` and `a1b2` are identifiers. `Ready?Set` is one identifier too: a mark does not end a word.

 **PARTLY IMPLEMENTED.** The `?` mark works; `!` is refused as an unexpected character wherever it appears. See DEF-03. The conformance program below therefore exercises `?` alone — the `!` half is a commitment the implementation does not yet meet, and pinning it there would only make the corpus red.


```

interpreter  compiler/Scanner.a24  ScanIdentifier
unit         Scan Identifier
unit         Scan Identifier With A Question Mark
conformance  0010-identifier-forms.a24

```

[LEX-008] An identifier mark may not **begin** an identifier. `?` and `!` alone are not identifiers, and neither are `?abc` and `!abc`: the mark is refused as an unexpected character [SRC-002], because nothing else in the language claims it.

⚠ NOT YET IMPLEMENTED. The implementation classes `?` as a letter, so it leads an identifier and `var ? := 7;` declares a variable; and it does not admit `!` in an identifier at all. See DEF-03.

```

interpreter  compiler/Scanner.a24  IsAlpha
defect       DEF-03-identifier-marks.a24
defect       DEF-03b-bang-is-not-an-identifier-mark.a24

```

The rule exists so that `Gate?` can be one word without `?` also becoming a name. A trailing mark reads as punctuation on a word; a leading one reads as an operator the language does not have.

[LEX-009] An identifier may not be spelled the same as a keyword in any case, because the keyword is recognised first. `var begin := 7;` and `var BEGIN := 7;` are both refused with `Expect variable name.`

```

interpreter  compiler/Scanner.a24  ScanIdentifier
unit         Scan Keywords
refusal      0003-keyword-is-not-a-name.a24

```

4.4 Keywords

[LEX-010] The following 37 words are keywords and are matched case-insensitively per [SRC-010]:

and	as	begin	break	case	class	const	constructor
do	else	end	except	exit	false	for	function
if	in	is	nil	not	object	of	or
private	procedure		public	raise	super	then	
this	true	try	type	uses	var	while	

No other word is a keyword. Every word not in this list is an identifier and may be declared as a name.

⚠ NOT YET IMPLEMENTED. The implementation registers one further word:

```

print

```

`print` is a keyword there, and it introduces a statement [STM-022]. Neither is part of the language; `var print := 7;` must be a declaration and is refused. See DEF-04.

```
interpreter  compiler/Scanner.a24  Keywords
unit         Scan Keywords
defect       DEF-04-print-is-a-keyword.a24
```

[LEX-011] `unit`, `test` and `on` are **not** keywords. They are ordinary identifiers that the grammar recognises by position — `unit` opening a file, `test` before a block's quoted name, `on` introducing a handler — and each may be used as a variable name.

```
interpreter  compiler/Scanner.a24  Keywords
interpreter  compiler/Parser.a24   UnitHeader
unit         Parse On Is Not A Keyword
conformance  0018-context-sensitive-words.a24
```

`var test := 7;` is a valid declaration, and so are the `unit` and `on` forms. Verified in all three.

4.5 Operators and punctuation

[LEX-012] The following are operators and punctuation:

```
( ) [ ] , . ; :
+ - * / = :=
< <= > >= <>
```

```
interpreter  compiler/Scanner.a24  ScanToken
unit         Scan Operators
conformance  0012-operators.a24
```

[LEX-013] `=` is equality and `:=` is assignment. Inequality is `<>`. There is no `==`, no `!=` and no `!` operator: negation is `not`, and `!` is an identifier mark [SRC-005] rather than punctuation.

```
interpreter  compiler/Scanner.a24  ScanToken
interpreter  compiler/TokenType.a24 TOKEN_ASSIGN
unit         Scan Operators
conformance  0012-operators.a24
```

Because `!` never begins a token [LEX-008], `A!=B` is unambiguous: it is the identifier `A!` compared with `B`. There is no `!=` for it to be mistaken for.

[LEX-014] `and` , `or` , `not` , `in` , `is` and `as` are operators spelled as keywords rather than punctuation, and are subject to [SRC-010].

```
interpreter  compiler/Scanner.a24  Keywords
unit         Scan Keywords
conformance  0012-operators.a24
```

⚠ `{` and `}` are not tokens of the language at all — not as comment delimiters, not as block delimiters, and not as set constructors. A block is `begin ... end` ; a collection literal uses `[` and `]` .

4.6 Integer literals

[LEX-015] An integer literal is a run of decimal digits.

```
integer_lit = decimal_digit { decimal_digit } .
```

```
interpreter  compiler/Scanner.a24  ScanNumber
unit         Scan Number
unit         Scan Integer Is Not A Double
conformance  0013-integer-literals.a24
```

[LEX-016] Decimal is the only base. There is no hexadecimal, octal or binary form, and no digit separator. `0x10` is not a number: it scans as the integer `0` followed by the identifier `x10` , and the two adjacent expressions then fail to parse.

⚠ **PLANNED — a later generation.** Hexadecimal, octal and binary forms and a digit separator are intended, and will change this rule. They are not part of the language described here. See Annex H.

```
interpreter  compiler/Scanner.a24  ScanNumber
refusal      0006-hex-is-not-a-literal.a24
```

[LEX-017] Leading zeros are permitted and carry no meaning. `007` is the integer 7, not an octal.

```
interpreter  compiler/Scanner.a24  ScanNumber
conformance  0013-integer-literals.a24
```

[LEX-018] An Integer is a signed 32-bit value, from `-2147483648` to `2147483647`. An arithmetic operation whose result falls outside that range **raises** rather than wrapping.

⚠ **NOT YET IMPLEMENTED.** Both processors wrap silently, so `2147483647 + 1` is `-2147483648` and a program can compute a wrong answer with no sign of it. See DEF-05.

```

interpreter  compiler/Interpreter.a24  VisitBinary
compiler     bootstrap/algol.c        alg_add
defect       DEF-05-integer-overflow-is-silent.a24

```

[LEX-033] An integer **literal** outside the range of [LEX-018] is refused when the program is read, not when it runs. It is a value the source states plainly and the processor can check without executing anything.

⚠ NOT YET IMPLEMENTED. The literal wraps silently, so `2147483648` is the integer `-2147483648` and `9999999999999999` is `276447231`. See DEF-05.

```

interpreter  compiler/Scanner.a24  ToInteger
defect       DEF-05-integer-overflow-is-silent.a24

```

The two halves are separated because they cost differently. A literal is checked once, while it is being scanned; an arithmetic result must be checked on every operation the program performs.

[LEX-019] There is no negative literal. A leading `-` is the unary operator applied to a non-negative literal, which is why `2-1` is a subtraction rather than two adjacent expressions.

```

interpreter  compiler/Scanner.a24  ScanToken
conformance  0013-integer-literals.a24

```

4.7 Double literals

[LEX-020] A double literal requires at least one digit on **both** sides of the point.

```
double_lit = decimal_digit { decimal_digit } "." decimal_digit { decimal_digit } .
```

```

interpreter  compiler/Scanner.a24  ScanNumber
unit         Scan Number Decimal
conformance  0014-double-literals.a24

```

[LEX-021] `1.` is therefore not a double. It is the integer `1` followed by the `.` operator, and a program containing it fails with `Expect property name after '.'`. Likewise `.5` is not a literal at all.

```

interpreter  compiler/Scanner.a24  ScanNumber
unit         Scan Integer Then Dot
refusal      0005-trailing-dot-is-not-a-double.a24

```

[LEX-022] There is no exponent notation. `1e5` scans as the integer `1` followed by the identifier `e5`, and the two adjacent expressions then fail to parse.

⚠ The language therefore **prints a form it cannot read**. `Str` renders a large `Double` in exponent notation — `1.0E300` — and that text is not a literal. Nothing becomes unreachable, because `Val` *does* parse the exponent form and `Val(Str(X))` round-trips; but a value cannot be written into a program the way the program writes it out.

⚠ **PLANNED — a later generation.** An exponent form is intended, and will change this rule. See Annex H.

```
interpreter  compiler/Scanner.a24  ScanNumber
refusal      0007-exponent-is-not-a-literal.a24
```

4.8 Character literals

[LEX-023] A quoted literal enclosing exactly one character **of source** is a `Char` rather than a `String`. `'a'` and `'é'` are both `Chars`.

⚠ **The measurement is taken on the source span between the quotes, not on the value**, and the distinction is load-bearing. `''''` is a doubled quote — two characters of source, one character of value — and it is a `String` [LEX-029]. Measuring the value would silently reclassify it.

⚠ **NOT YET IMPLEMENTED.** The implementation measures the span in *bytes*, so `'é'` is a `String` of length 2. See DEF-01.

```
interpreter  compiler/Scanner.a24  ScanString
unit         Scan One Character Is A Char
defect       DEF-01-text-is-bytes.a24
```

[LEX-024] A `Char` may also be written `#` followed by decimal digits, giving the character with that code point: `#65` is `A`, `#10` is a line feed and `#233` is `é`. A `#` not followed by a digit is an error reading [line N] Error: Invalid character: `C`.

```
char_lit = "" source_character "" | "#" decimal_digit { decimal_digit } .
```

```
interpreter  compiler/Scanner.a24  ScanChar
unit         Scan Char By Code Point
unit         Scan Char Without Digits
conformance  0015-char-literals.a24
```

[LEX-025] A `Char` is a Unicode code point: `0 ... 10FFFF`, excluding the surrogate range `D800 ... DFFF`, which encodes no character. A `#` literal outside that range is refused when the program is read, with the shape every other scan error has — [line N] Error: ... — because that is where it is detected.

⚠ **NOT YET IMPLEMENTED.** Two ways. The range is 0 ... 127, so `#200` is refused where the specification admits it; and the refusal is raised rather than recorded, so it arrives as `Uncaught: Char is limited to 0..127.` with no line number and no source caret, unlike every other scan error [ERR-004]. See DEF-06.

```
interpreter  compiler/Interpreter.a24  CharNative
compiler     bootstrap/algol.c         alg_char
defect       DEF-06-char-range-and-diagnostic.a24
```

`ScanChar` builds the value by calling the `Char` built-in during the scan (`compiler/Scanner.a24`), which is why a range failure surfaces as a runtime raise from inside the scanner rather than as a scan error.

[LEX-026] A `Char` and a `String` are never equal, however alike they look. `'a' = 'a'` is true because both sides are `Chars`; `Copy('abc', 0, 1) = 'a'` is **false**, because `Copy` yields a `String` of length one and the `Char` `'a'` is not it.

```
interpreter  compiler/Interpreter.a24  IsEqual
compiler     bootstrap/algol.c         equals
conformance  0015-char-literals.a24
```

4.9 String literals

[LEX-027] A string literal is enclosed in single quotes. A quote within it is written twice.

```
string_lit = "'" { source_character_other_than_quote | "'" } "'" .
```

```
interpreter  compiler/Scanner.a24  ScanString
unit         Scan String
unit         Scan Doubled Quote
conformance  0016-string-literals.a24
```

[LEX-028] There are no backslash escapes. `'a\nb'` is four characters, and its element at index 1 is the backslash itself. A line feed is written `#10` and concatenated.

```
interpreter  compiler/Scanner.a24  ScanString
conformance  0016-string-literals.a24
```

[LEX-029] `''` is the empty `String`. `''''` is a `String` of length one holding a quote — **not** a `Char`, because [LEX-023] measures the source span, which is two characters.

```

interpreter  compiler/Scanner.a24  ScanString
unit         Scan Empty String
unit         Scan An Escaped Quote Is A String
conformance  0016-string-literals.a24

```

[LEX-030] A string literal may span lines. The line feed is part of its value and advances the line count, so 'one ↵ two' is seven characters.

```

interpreter  compiler/Scanner.a24  ScanString
conformance  0016-string-literals.a24

```

[LEX-031] A string that reaches the end of the file unclosed is an error reading [line N] Error: Unterminated string. , where N is the line the string **opened** on.

⚠ **NOT YET IMPLEMENTED.** N is the line the scan *reached*, which is the last line of the file. On a large source the two differ by however much text follows the stray quote, and the line reported is the one place the fault certainly is not. See DEF-07.

```

interpreter  compiler/Scanner.a24  ScanString
unit         Scan Unterminated String
defect       DEF-07-unterminated-string-line.a24

```

[LEX-032] #0 is not a Char. A code point of 0 is refused exactly as an out-of-range one is [LEX-025], when the program is read.

⚠ **NOT YET IMPLEMENTED.** #0 is accepted and #0 is Char is true, but a String cannot hold it: concatenating one truncates the String there, so Length('a' + Str(#0) + 'b') is 2. Both processors agree, because both represent a String as C does, terminated by the zero byte. See DEF-08.

```

interpreter  compiler/Interpreter.a24  StrNative
compiler     bootstrap/algol.c         alg_string
defect       DEF-08-nul-char-truncates.a24

```

Refusing #0 is the smaller of the two available fixes and matches the range check that already exists. The larger one — giving a String an explicit length so it can hold a zero character — is the better language, and [SRC-001] already obliges a String to carry a character count distinct from its byte length, so the two changes meet. This rule is written so that adopting the larger fix later relaxes a restriction rather than reversing a guarantee.

5. Constants and variables

5.1 Variable declarations

[VAR-001] A variable is introduced by `var` , with an optional declared type and an optional initializer.

```
VarDecl = "var" identifier [ ":" Type ] [ "!=" Expression ] ";" .  
Type    = identifier [ "of" identifier ] .
```

```
interpreter  compiler/Parser.a24  VarDeclaration  
unit        Parse Var Statement  
unit        Parse Var Expect Semicolon  
conformance 0019-declaration-forms.a24
```

[VAR-002] A variable declared without an initializer holds `nil` , whatever its declared type. **There is no zero value:** an uninitialized `Integer` is `nil` , not `0` .

```
interpreter  compiler/Interpreter.a24  VisitVarStmt  
conformance 0022-no-zero-value.a24
```

[VAR-003] A declared type constrains the initializer and every later assignment. A violation is the error `Type mismatch!`

```
interpreter  compiler/TypeChecker.a24  Assignable  
refusal     0008-declared-type-constrains.a24
```

[VAR-004] A value **widens** to reach a written type, and is converted at the point it arrives. There are two widening pairs:

From	To	Example
Integer	Double	<code>var X : Double := 1;</code> gives <code>X</code> the Double <code>1.0</code>
Char	String	<code>var S : String := 'a';</code> gives <code>S</code> a String of length 1

The variable holds the wider type afterwards. A declaration never misdescribes what it holds.

⚠ NOT YET IMPLEMENTED. Neither widening is applied where a type is written: `var X : Double := 1;` , `var S : String := 'a';` and an `Exit 1` from a function declared `: Double` are each `Type mismatch!` See DEF-10.

⚠ The rules already exist; they are simply not applied here. `1 + 1.5` is `2.5` , `'a' + 'bc'` is `abc` , and `'a' + 'b'` is a String of two characters — the operators widen both pairs today. It is only the paths carrying a *written* type that refuse, so a declared type currently means something **narrower** than the operators

do. That inconsistency is the defect; the widening itself is settled behaviour everywhere else, which is why this is a fault to be fixed rather than a facility the language has yet to gain.

```
interpreter  compiler/TypeChecker.a24  Assignable
conformance 0025-operators-widen.a24
defect      DEF-10-widening-is-refused.a24
```

[VAR-017] Widening applies wherever a value meets a written type — the six **assignment contexts**, and nowhere else:

```
var X : T := E ;      const X : T := E ;
X := E ;              Obj.Field := E ;
Exit E ;              (against a declared return type)
F (E) ;               (against a declared parameter type)
```

 **Comparison is not among them and does not widen.** 'a' and Copy('abc', 0, 1) remain unequal [LEX-026]. A widening converts *toward a target type*, and an = supplies none — it would have to invent one, and "convert when the sides differ" is the rule that makes = unpredictable. See D-6, which weighs the same question for membership.

```
interpreter  compiler/TypeChecker.a24  Assignable
defect      DEF-10-widening-is-refused.a24
```

[VAR-018] Narrowing is refused in every one of those contexts. var X : Integer := 1.5; is a mismatch: the value does not fit, and choosing how to lose the fraction is not a decision a declaration should make silently. var C : Char := 'ab'; is refused for the same reason.

```
interpreter  compiler/TypeChecker.a24  Assignable
refusal     0014-no-implicit-narrowing.a24
```

[VAR-005] nil satisfies every declared type, so var X : Integer := nil; is accepted.

```
interpreter  compiler/TypeChecker.a24  Assignable
compiler     bootstrap/algol.c         alg_is
conformance  0022-no-zero-value.a24
```

[VAR-006] Any is the declared type meaning *not known*. A variable declared Any accepts every value.

It does **not** flow the other way. A value whose type is Any, or whose type could not be determined at all, does not satisfy a written type: neither var I : Integer := A; nor a later I := A; is accepted where A is Any. The conversion must be written, and as [VAL-007] is how to write it.

⚠️ **A type is not a suggestion once it is written.** Where a program declares a type, every value reaching that variable either has the type already or is converted by an expression that says so and checks it. This is what lets a declared type be relied upon — by a reader, and by the C back end, which can only generate a machine representation for a variable whose type it may trust.

⚠️ Writing no type remains entirely permissive. `var A := M.Get (1);` is ordinary and unremarkable; the rule bites only where a type was written down.

⚠️ **PARTLY IMPLEMENTED.** A declaration enforces this and a later assignment does not, so `var I : Integer := A;` is refused while `var I : Integer;` followed by `I := A;` is accepted — the same value reaching the same variable one line later. See DEF-09.

```
interpreter  compiler/TypeChecker.a24  Assignable
conformance  0020-any-accepts-every-value.a24
defect       DEF-09-assignment-escapes-the-type.a24
```

Assignable itself permits Any in both directions; the strictness lives in an extra check on the declaration (compiler/TypeChecker.a24 , MapType). Bringing the two paths together is what DEF-09 asks for.

[VAR-007] A name may not be declared twice in one scope. The second is refused with 'X' is already defined.

```
interpreter  compiler/Resolver.a24  CheckDuplicates
unit         Resolve Duplicate Variable
refusal      0009-no-redeclaration.a24
```

[VAR-008] A collection may carry an element type, written `of : var L : List of Integer := [];` . Every collection type accepts one — List , Map , Set and Array .

⚠️ **PARTLY IMPLEMENTED.** `of` parses only after List . Map of Integer , Set of Integer and Array of Integer are each refused with Expect ';' after variable declaration. See DEF-11.

```
interpreter  compiler/Parser.a24  VarDeclaration
conformance  0019-declaration-forms.a24
defect       DEF-11-of-is-list-only.a24
```

[VAR-016] An element type is a **source of types for reads, and no constraint on writes**. Given `var L : List of Integer :`

Expression	Type
<code>L[0]</code>	Integer

Expression	Type
<code>X in for var X in L</code>	Integer
<code>L.Add (V)</code>	accepts any <code>V</code> ; the element type is not checked

⚠ The asymmetry is deliberate rather than an oversight to be tidied away. The reading half is what makes a declared element type worth writing — it is how a loop variable acquires a type without one being written on it — and it costs nothing at run time. Checking every insertion is a different and much larger commitment; see Annex H.

```
interpreter  compiler/TypeChecker.a24  Reduce
conformance 0021-element-types-flow-to-reads.a24
```

5.2 Sections

[VAR-009] In the header of a function, procedure, class or object, `var` may open a **section**: a run of declarations, each ending in `;`, closed by the next section marker or by `begin`.

```
VarSection = "var" { identifier { "," identifier } [ ":" Type ] [ "!=" Expression ] ";" } .
```

```
interpreter  compiler/Parser.a24  DeclarationSection
unit         Parse A Function Local Var Section
conformance  0023-sections.a24
```

[VAR-010] Within a section several names may share one declaration: `A, B : Integer;` declares both.

```
interpreter  compiler/Parser.a24  DeclarationSection
unit         Parse A Comma Group Stays A Group
unit         Parse A Single Name Is Not A Group
conformance  0023-sections.a24
```

[VAR-011] A section is a feature of a **header**, not of a program body. At the top level `var` declares exactly one name, and a run of declarations beneath it is read as ordinary statements — `var A : Integer;` followed by `B : String;` fails on the second with `Expect ';' after expression`.

```
interpreter  compiler/Parser.a24  ReadDeclarationSections
refusal      0010-no-section-at-top-level.a24
```

5.3 Constants

[VAR-012] A constant is introduced by `const` and **must** be given a value. Omitting the initializer is refused with A constant must be given a value.

```
ConstDecl = "const" identifier [ ":" Type ] "!=" Expression ";" .
```

```
interpreter  compiler/Parser.a24  ConstDeclaration
unit        Parse A Constant Must Be Given A Value
refusal     0011-const-must-be-given-a-value.a24
```

[VAR-013] A constant may not be assigned to. The attempt is refused with Can't assign to constant 'C'.

```
interpreter  compiler/Resolver.a24  VisitAssignExpr
refusal     0012-const-is-not-assignable.a24
```

[VAR-014] ⚠ A constant's initializer is an **ordinary expression evaluated at run time**, not a constant expression. `const C := 1 + 2;` is legal, and so is `const C := V;` where `V` is a variable — the constant takes whatever value `V` held at that moment.

`const` therefore means *this binding may not be reassigned*, not *this value is known before the program runs*.

```
interpreter  compiler/Interpreter.a24  VisitVarStmt
conformance  0024-const-is-a-runtime-expression.a24
```

[VAR-015] `const` may open a section on the same terms as `var`, and the two may appear together in one header.

```
interpreter  compiler/Parser.a24  DeclarationSection
unit        Parse A Const Section
unit        Parse Var And Const Sections Together
conformance  0023-sections.a24
```

6. Types

6.1 The kinds of type

[TYP-001] A value has exactly one of these runtime types:

Kind	Types
------	-------

Kind	Types
Primitive	Integer , Double , String , Char , Boolean
Declared	a class type, an enumeration type
Collection	List , Set , Stack , Array , Map
Resource	Buffer , TextFile
Unknown	Any — a declaration only, never a runtime type

```

interpreter  compiler/ObjFunction.a24  TypeNameOf
compiler     bootstrap/algol.c        alg_is
conformance  0026-the-runtime-types.a24

```

[TYP-002] A type is written as an identifier. The only compound form is a collection type with an element type, written of — List of Integer , Map of Token [VAR-008].

```
Type = identifier [ "of" identifier ] .
```

```

interpreter  compiler/Parser.a24  VarDeclaration
conformance  0026-the-runtime-types.a24

```

[TYP-003] Char and String are distinct types, and a one-character value is a Char . 's' is String is **false**.

⚠ Widening does not change this. A Char widens to a String on its way into a written type [VAR-004], but `is` asks what a value *is*, not what it could become, and a type test is not one of the six assignment contexts [VAR-017]. A Char that has widened is a String and answers so; the Char it came from is not.

```

interpreter  compiler/ObjFunction.a24  TypeNameOf
conformance  0026-the-runtime-types.a24

```

6.2 Any

[TYP-004] Any is the declared type meaning *the type is not known*. A variable declared Any accepts every value; a value of type Any does not satisfy a written type without a cast [VAR-006]. No value ever reports Any as its runtime type, so `X is Any` is false for every X [VAL-005].

```

interpreter  compiler/TypeChecker.a24  Assignable
conformance  0020-any-accepts-every-value.a24
refusal      0013-any-does-not-satisfy-a-written-type.a24

```

The asymmetry is the point of [VAR-006]: Any `is` is where a type is not known, and a written type is a claim that it is. Moving from the first to the second is a conversion, and `as` is how one is written.

6.3 nil

[TYP-005] `nil` is of no type at all. `nil is T` is **false** for every `T`, including the type `nil` was declared as.

```
interpreter  compiler/Interpreter.a24  VisitIsExpr
compiler     bootstrap/algol.c        alg_is
conformance  0027-nil-has-no-type.a24
```

[TYP-006] `nil` nonetheless satisfies every declared type for the purpose of assignment — see [VAR-005]. A value that is not there has no type to check, and is accepted everywhere.

```
interpreter  compiler/TypeChecker.a24  Assignable
conformance  0027-nil-has-no-type.a24
```

[TYP-013] A type name must denote a declared type. A name that denotes nothing is refused when the program is read, rather than being read as a type no value has.

⚠ **NOT YET IMPLEMENTED.** The name is never resolved: `1 is Nonexistent` is `false`, silently. A misspelled type answers `false` and the branch it guards never runs — while an undefined *variable* in the same position is `Undefined variable 'X'`, so `is` is uniquely permissive about its right-hand side. See DEF-13.

```
interpreter  compiler/Interpreter.a24  VisitIsExpr
defect       DEF-13-unknown-type-name-is-silent.a24
```

This is not a gradual-typing case. Gradual typing concerns a *value* whose type is not known, which is ordinary; a type *name* is written by the programmer and must denote something.

6.4 Collection types

[TYP-007] The five collection types are distinguished by their kind, and each answers `is` to its own name only. A `List` is not a `Set`.

```
interpreter  compiler/ObjCollection.a24  Kind
conformance  0028-collection-types-are-distinct.a24
```

[TYP-008] `Array` is fixed in size. Its elements begin as `nil`, it is indexed from zero, and an index outside its bounds is the runtime error `Index N out of range 0..M`. It does not grow on assignment.

```
interpreter  compiler/ObjCollection.a24  At
compiler     bootstrap/algol.c          alg_subscript_set
conformance  0029-array-is-fixed.a24
```

[TYP-009] A collection is **not** a class instance. It has no `ClassName`, and asking for one is the error `Undefined property 'ClassName'`.

```
interpreter  compiler/ObjCollection.a24  Get
conformance  0030-collections-have-no-classname.a24
```

6.5 What a class type cannot do

These rules are normative in their own right, and together they say which built-in behaviour a program cannot reproduce for a type of its own. Annex E takes up what it would cost to lift each one.

[TYP-010] A class instance may not be subscripted. `B[0]` on an instance is the runtime error `Subscript target should be an ordinal`, whatever methods the class declares.

 **PLANNED — a later generation.** A subscript operator a class may declare. See Annex H, H-4.

```
interpreter  compiler/Interpreter.a24  VisitSubscriptExpr
conformance  0031-instance-is-not-subscriptable.a24
```

[TYP-011] A class instance may not be iterated. `for var X in B do` over an instance is the runtime error `Can only iterate a collection or a String`.

 **PLANNED — a later generation.** An iteration protocol a class may implement. See Annex H, H-5.

```
interpreter  compiler/Interpreter.a24  VisitForInStmt
conformance  0032-instance-is-not-iterable.a24
```

[TYP-012] A class exposes a **field** without parentheses and a **method** with them. There is no getter declaration, so a computed value cannot be read as a property: a method named `Length` read as `B.Length` yields the function itself, printing `<fn Length>`, where a collection's `Length` yields its count.

 **PLANNED — a later generation.** A computed property — a method read without parentheses. See Annex H, H-6.

```
interpreter  compiler/ObjInstance.a24  Get
conformance  0033-no-computed-property.a24
```

7. Properties of types and values

7.1 Assignability

[VAL-001] A value is assignable to a declaration when its type is the declared type, when the **declaration** is `Any`, when the value is `nil`, when its class inherits from the declared class, or when it widens to the declared type [VAR-004].

⚠ A value *of type* `Any` is not assignable to a written type; only the reverse holds [VAR-006].

```
interpreter  compiler/TypeChecker.a24  Assignable
conformance 0034-assignability.a24
defect       DEF-10-widening-is-refused.a24
```

[VAL-002] Nothing else converts. The widenings are exactly the two of [VAR-004] — Integer to Double and Char to String — and they apply only at the assignment contexts of [VAR-017]. There is no narrowing, no conversion between a number and a String, and no user-defined conversion.

```
interpreter  compiler/TypeChecker.a24  Assignable
conformance 0034-assignability.a24
refusal      0014-no-implicit-narrowing.a24
```

7.2 Type tests

[VAL-003] `X is T` is true when the runtime type of `X` is `T`, or is a class inheriting from `T`. Inheritance is followed upward only: a `Dog` is an `Animal`, and an `Animal` is not a `Dog`.

```
interpreter  compiler/Interpreter.a24  VisitIsExpr
compiler     bootstrap/algol.c         alg_is
conformance  0035-type-tests.a24
```

[VAL-004] `nil is T` is **false** for every `T`. A value that is not there has no type to test.

```
interpreter  compiler/Interpreter.a24  VisitIsExpr
compiler     bootstrap/algol.c         alg_is
conformance  0035-type-tests.a24
```

[VAL-005] `X is Any` is **false** for every `X`. `Any` is a declaration, never a runtime type.

⚠ `Any` is nonetheless a legal type name in `is` [TYP-013]. It denotes something; it just never matches.


```
interpreter  compiler/Interpreter.a24  VisitIsExpr
conformance 0035-type-tests.a24
```

[VAL-006] The type name in `is` is matched case-insensitively, so `1 is integer` is true. This is [SRC-011] applied to a type name and is not a special case.

```
interpreter  compiler/Interpreter.a24  VisitIsExpr
conformance 0035-type-tests.a24
```

7.3 Casts

[VAL-007] `X as T` is a **checked conversion**. It tells the checker that the expression has type `T`, and verifies the claim when the program runs: if `X` is not a `T`, the cast raises.

`nil` satisfies every type [VAR-005] and therefore passes every cast.

⚠ `as` is the one construct that moves a value from untyped into typed [VAR-006], so it carries the whole weight of that boundary. A cast that could not fail would make the boundary a formality, and every declared type downstream of it a claim nothing had checked.

⚠ **NOT YET IMPLEMENTED.** The cast has no runtime effect at all: `X as Integer` where `X` holds `'text'` yields `'text'` and raises nothing. See DEF-12.

```
interpreter  compiler/TypeChecker.a24  Reduce
defect      DEF-12-as-is-unchecked.a24
```

7.4 Truthiness

[VAL-008] A value is **falsey** if it is `nil`, `False`, the Integer `0`, or an enumeration member whose ordinal is `0`. Every other value is **truthy**.

⚠ In particular these are all truthy: `0.0`, the empty String `''`, the empty List `[]`, the empty Map `[:]`, and every `Char`.

⚠ Truthiness is **independent of a value's contents**. A collection is a thing, and a thing is there; `if not S` then `therefore` does not test emptiness, and `if S.Length = 0` then `is` is the only spelling that does.

```
interpreter  compiler/Interpreter.a24  IsTruthy
compiler     bootstrap/algol.c        alg_truthy
unit         Execute Logical Truthy
unit         Evaluate Unary Bang Nil
conformance 0036-truthiness.a24
```

D-8 observes that `Integer 0` is the odd one out, and that were it truthy the rule would reduce to "only `nil` and `False` are false". That is a change to what existing programs mean and is not scheduled; the rule as written is the decided one.

7.5 Equality

[VAL-009] `=` and `<>` promote numerically. `1 = 1.0` is true, and so is `0 = 0.0`.

```
interpreter  compiler/Interpreter.a24  IsEqual
compiler     bootstrap/algol.c        equals
unit         Evaluate Binary Equal Equal
conformance  0037-equality.a24
```

[VAL-010] A `Char` is never equal to a `String` — see [LEX-026]. `'a'` and `Copy('abc', 0, 1)` are not equal.

⚠ **Widening does not reach equality**, and the asymmetry with [VAL-009] is deliberate. A numeric promotion has one obvious target — the wider of the two types — and converting toward it loses nothing. `Char` against `String` has no target at all: it is a change of representation rather than a widening of value, and picking a direction would mean `=` converting its operands differently depending on which side they arrived on.

⚠ The rule bites less often than it appears to. `'a' = 'a'` is already true, because both sides are `Chars`; it is only reached when one side came from `Copy`, `Str` or a subscript. The complaint worth acting on is how easily a one-character `String` is produced by accident, not how `=` treats one.

```
interpreter  compiler/Interpreter.a24  IsEqual
conformance  0037-equality.a24
```

[VAL-011] Class instances, collections and enumeration members compare by **identity**, not by contents. `[1, 2] = [1, 2]` is **false**: they are two collections. Two references to one collection are equal.

⚠ There is no way for a class to say otherwise. Comparing by contents needs an operator the program may define, which the language does not have; see Annex H, H-8.

```
interpreter  compiler/Interpreter.a24  IsEqual
compiler     bootstrap/algol.c        equals
conformance  0037-equality.a24
```

[VAL-012] `nil = nil` is true.

```
interpreter  compiler/Interpreter.a24  IsEqual
unit         Evaluate Binary Bang Equal Nil
conformance  0037-equality.a24
```

7.6 Membership

[VAL-013] `in` , `Contains` and `Map` key lookup use the equality of [VAL-009]. Membership and equality are one relation: if $X = Y$ then a collection holding Y contains X . `1 in [1.0]` is **true**, and a `Map` holding the key `1` contains the key `1.0` .

⚠ NOT YET IMPLEMENTED. All three compare strictly, with no numeric promotion, so `1 in [1.0]` and `1.0 in [1]` are both false and `Keys.Contains(1.0)` is false for a `Map` holding `1` — while `1 = 1.0` is true. A program can hold two values it calls equal and find only one of them. See DEF-14.

```
interpreter  compiler/ObjCollection.a24  Invoke
compiler     bootstrap/algol.c           strict_equals
defect       DEF-14-membership-does-not-follow-equality.a24
```

Each half was defensible alone, which is how they came to disagree: `=` promotes because arithmetic does, and membership is strict because a hash table cannot be built over a relation that promotes. The second reason is a statement about the implementation, not about the language, and it is the one that gives way — see D-6 for the cost.

7.7 Ordering

[VAL-014] `<` , `<=` , `>` and `>=` apply to numbers and to `Char` only. **Strings are not ordered:** `'ab' < 'cd'` is the runtime error `Operands must be numbers` .

⚠ PLANNED — a later generation. Ordering for Strings. See Annex H, H-7.

```
interpreter  compiler/Interpreter.a24  VisitBinary
unit         Evaluate Binary Greater Left Not Number
conformance  0038-strings-are-not-ordered.a24
```

A program needing to order text must compare it character by character, which is what `compiler/CEmitter.a24` 's `TextLess` does — a function the compiler writes for itself because the language does not provide the operator.

8. Declarations and scope

8.1 Blocks

[DCL-001] A block is `begin ... end` and introduces a scope. A name declared inside it is not visible after it: reading one is `Undefined variable 'X'` .

```
interpreter  compiler/Resolver.a24  BeginScope
conformance  0039-blocks-and-scope.a24
```

[DCL-002] A block sees every name of the scopes enclosing it.

```
interpreter  compiler/Resolver.a24  ResolveLocal
unit         Resolve One Hop
unit         Resolve Two Hops
conformance  0039-blocks-and-scope.a24
```

8.2 Shadowing

[DCL-003] A declaration shadows an outer one of the same name for the rest of its scope. The outer binding is untouched and reappears when the scope ends.

```
interpreter  compiler/Resolver.a24  Declare
unit         Resolve Same Level
conformance  0040-shadowing.a24
```

[DCL-004] A `var` may shadow a `const`. The inner name is an ordinary variable and may be assigned; the outer constant is unaffected.

```
interpreter  compiler/Resolver.a24  DeclareBinding
conformance  0040-shadowing.a24
```

8.3 Declaration and use

[DCL-005] A local may not be read in its own initializer. `var X := X;` inside a block is refused with `Can't read local variable in its own initializer.`, even where an outer `X` exists.

```
interpreter  compiler/Resolver.a24  ResolveLocal
unit         Resolve Local Variable Is Own Initializer
refusal      0015-own-initializer.a24
```

[DCL-006] A **function or class** declared at the top level of a file is visible throughout that file, wherever it is written. A call may precede the declaration, so a program may be organized from the top down.

 **NOT YET IMPLEMENTED.** The interpreter creates the binding when the declaration runs, so a call above it is `Undefined variable 'F'`. The compiled back end already does what this rule requires. See DEF-15.

```
interpreter  compiler/Interpreter.a24  VisitFunctionStmt
defect       DEF-15-declarations-are-not-hoisted.a24
```

[DCL-016] A **variable or constant** is not visible before its declaration has run. Its initializer is an expression evaluated in order [VAR-014], and a name read before that has no value to give — so it is an error, not `nil` .

⚠ The split is deliberate. A function or class declaration is complete as soon as it is read and has nothing to execute; a `var` has an initializer whose effects belong at the point it is written. Hoisting the first is what lets a file be read top-down; hoisting the second would silently substitute `nil` for a value that does not exist yet.

```
interpreter  compiler/Interpreter.a24  VisitVarStmt
conformance 0044-variables-are-not-hoisted.a24
```

[DCL-007] A free name in a function body is resolved **when the body runs**, not where it is written. Two functions may therefore call each other, provided neither is called before both declarations have run.

```
interpreter  compiler/Interpreter.a24  LookupVariable
conformance 0041-mutual-recursion.a24
```

This is why mutual recursion works even under the current implementation, where [DCL-006] does not: the *call* inside a body is resolved late, so only a call at the top level, above the declaration, meets the missing binding.

8.4 Loop variables

[DCL-008] A variable declared in a `for` header belongs to the loop, in both forms, and is not visible after it ends.

For the counted form this follows from [DCL-001] rather than being a rule of its own: `for` **desugars into a block** holding the initializer and a `while` , so the variable is scoped because it is inside a block.

```
interpreter  compiler/Parser.a24    ForStatement
interpreter  compiler/Resolver.a24  VisitForInStmt
unit         Parse For Statement
conformance 0039-blocks-and-scope.a24
```

8.5 `this` and `super`

[DCL-009] `this` outside a class is refused with `Can't use 'this' outside a class.`

```
interpreter  compiler/Resolver.a24  VisitThisExpr
unit         This Is Never Caught
refusal      0016-this-outside-a-class.a24
```

[DCL-010] `super` outside a class is refused with `Can't use 'super' outside a class.` , and inside a class having no superclass with `Can't use 'super' in a class with no superclass.`

```
interpreter  compiler/Resolver.a24  VisitSuperExpr
refusal      0017-super-outside-a-class.a24
refusal      0018-super-with-no-superclass.a24
```

8.6 Visibility

[DCL-011] `private:` and `public:` are section markers within a class or object, each governing the members that follow it. A member declared under no marker is public.

```
interpreter  compiler/Parser.a24  ReadDeclarationSections
unit         A Public Member Is Reachable From Outside
unit         A Private Field Is Not Readable From Outside
conformance  0042-visibility.a24
refusal      0019-private-through-a-typed-receiver.a24
```

[DCL-012] The body starts public however the header ended. A `private:` in the header does not carry across `begin` .

```
interpreter  compiler/Parser.a24  ReadDeclarationSections
unit         The Body Starts Public However The Header Ended
conformance  0042-visibility.a24
```

[DCL-013] Privacy belongs to the **class**, not to the object. A method may reach the private members of another instance of its own class.

```
interpreter  compiler/TypeChecker.a24  CheckVisibility
unit         Another Instance Of The Same Class Reaches Its Privates
conformance  0042-visibility.a24
```

[DCL-014] A subclass does not reach what its parent hid. Reading a parent's private member through a receiver declared as the parent is refused with `'N' is private to P.`

```
interpreter  compiler/TypeChecker.a24  CheckVisibility
unit         A Subclass Does Not Reach What Its Parent Hid
refusal      0020-subclass-does-not-reach-what-a-parent-hid.a24
```

[DCL-015]  `private:` **is advisory**. It is checked **statically, and only where the receiver's type is known**. Reached through a receiver declared `Any` , or through a bare name inside a method — which resolves through `this` , and `this` has no type — a private member is readable and writable from anywhere.

```

var C : Any := Counter ();
WriteLn (C.Count);      // the private field, read
C.Count := 99;          // and written

```

This is normative: `private:` states an intention and buys a diagnostic wherever types are written down. It is **not** a boundary, and a program must not rely on it as one. Both processors agree, so it is a property of the language rather than of one implementation.

⚠ **The guarantee is therefore strongest exactly where it is least needed** — in well-annotated code — and absent from the code most likely to be reaching somewhere it should not. That is an honest description of a checker in a gradually typed language, not an accident, and D-9 records what enforcing it would cost.

⚠ `as` becoming a checked conversion [VAL-007] does not close this. Member access is not one of the assignment contexts [VAR-017], so nothing obliges a receiver to be narrowed before it is read through.

```

interpreter  compiler/TypeChecker.a24  CheckVisibility
unit         A Private Member Is Caught Through A Declared Receiver
conformance  0043-visibility-is-advisory.a24

```

9. Expressions

9.1 Precedence and associativity

[EXP-001] Operators bind in this order, tightest first:

	Operators
1	<code>f(...)</code> call · <code>a[i]</code> subscript · <code>a.b</code> property
2	<code>-</code> unary · <code>not</code> · <code>as</code>
3	<code>*</code> · <code>/</code>
4	<code>+</code> · <code>-</code>
5	<code><</code> · <code><=</code> · <code>></code> · <code>>=</code> · <code>in</code> · <code>is</code>
6	<code>=</code> · <code><></code>
7	<code>and</code>
8	<code>or</code>
9	<code>:=</code>

```
interpreter  compiler/Parser.a24  Expression
unit        Parse Term Plus
unit        Parse Factor Star
conformance 0045-precedence.a24
```

$1 + 2 * 3$ is 7, $-2 * 3$ is -6, not True and False is false, True or False and False is true, and False = False and False is false. Each distinguishes its pair. Verified.

[EXP-002] Binary operators of one level are left-associative: $10 - 2 - 3$ is 5 and $12 / 2 / 3$ is 2.

```
interpreter  compiler/Parser.a24  Term
conformance 0045-precedence.a24
```

[EXP-003] `as` binds **tightly**, at the level of unary `-` and `not`. It applies to the operand beside it and to nothing further: `A and B as C` is `A and (B as C)`, and `A as Integer > 3` is `(A as Integer) > 3`.

⚠ **NOT YET IMPLEMENTED.** `as` binds looser than `or`, so `A and B as C` casts the whole conjunction, and `A as Integer > 3` does not parse at all — the cast consumes the expression and the comparison is left over, giving Expect `')` after arguments. See DEF-16.

⚠ The binding stopped being cosmetic when `as` became a checked conversion [VAL-007]. A cast covering a conjunction now **raises** when the conjunction is not of the named type, which is almost never what was written; while a cast had no runtime effect, the same reading was merely an oddity in the checker.

```
interpreter  compiler/Parser.a24  Expression
defect      DEF-16-as-binds-too-loosely.a24
```

9.2 Arithmetic

[EXP-004] Integer arithmetic yields an Integer, **including** `/`, which divides and truncates toward zero: $7 / 2$ is 3 and $-7 / 2$ is -3.

```
interpreter  compiler/Interpreter.a24  VisitBinary
compiler     bootstrap/algol.c         alg_divide
unit        Evaluate Binary Slash
conformance 0046-arithmetic.a24
```

[EXP-005] A Double on either side promotes the operation and the result: $7.0 / 2$ and $7 / 2.0$ are both 3.5, and $1 + 2.0$ is 3.0.

```
interpreter  compiler/Interpreter.a24  VisitBinary
unit        Evaluate Binary Plus Mixed
conformance 0046-arithmetic.a24
```


[EXP-006] ⚠ Integer division by zero is the runtime error `Division by zero`. **Double division by zero is not an error**: it yields `Infinity`, `-Infinity` or `NaN`, and the program continues.

⚠ Whether dividing by zero is a fault or a value therefore depends on which type reached the operator, and [EXP-005] promotes an Integer whenever it meets a Double — so an edit far from the division can move it from one category to the other. This is specified rather than merely tolerated: each behaviour is right for its own type. IEEE 754 defines the Double case and there is no integer infinity to return for the other.

```
interpreter  compiler/Interpreter.a24  VisitBinary
compiler     bootstrap/algol.c        alg_divide
conformance  0047-division-by-zero.a24
```

[EXP-007] Arithmetic that leaves the bounds of a 32-bit Integer **raises** — see [LEX-018].

⚠ **NOT YET IMPLEMENTED**. It wraps silently in both processors. See DEF-05.

```
compiler     bootstrap/algol.c  alg_add
defect       DEF-05-integer-overflow-is-silent.a24
```

9.3 Concatenation

[EXP-008] `+` concatenates when **either** operand is a String or a Char, converting the other. `'x' + 1` is `x1`, `1 + 'x'` is `1x`, and `'a' + 'b'` — two Chars — is the String `ab`.

```
interpreter  compiler/Interpreter.a24  VisitBinary
compiler     bootstrap/algol.c        alg_add
unit         Evaluate Binary Plus String
conformance  0025-operators-widen.a24
```

9.4 Logical operators

[EXP-009] `and` and `or` **short-circuit**. The right operand is evaluated only when the left does not decide the result.

```
interpreter  compiler/Interpreter.a24  VisitLogical
unit         Execute Logical And
unit         Execute Logical Or
conformance  0048-logical-operators.a24
```

[EXP-010] Both operators test truthiness [VAL-008] rather than requiring a Boolean.

```
interpreter  compiler/Interpreter.a24  IsTruthy
unit        Execute Logical Truthy
conformance 0048-logical-operators.a24
```

9.5 Calls

[EXP-011] A call checks arity. A mismatch is Expected N arguments but got M.

```
interpreter  compiler/Interpreter.a24  VisitCall
unit        Call Wrong Number Of Arguments
conformance 0049-call-failures.a24
```

[EXP-012] Calling something that is neither a function nor a class is Can only call functions and classes.

```
interpreter  compiler/Interpreter.a24  VisitCall
unit        Call Non Function
conformance 0049-call-failures.a24
```

[EXP-013] Where a name is overloaded, selection is made on the **whole signature** — the number of arguments and the type of each — and is made **at run time**, from the arguments actually passed.

⚠ **Run-time selection is required, not an implementation choice.** The type system is gradual, so an argument's declared type may be `Any` or absent while its value has a definite type: `var A : Any := 1;` passed to a name overloaded on `Integer` and `String` selects the `Integer`. No static rule could reach that, and an implementation must not resolve overloads at compile time on declared types alone.

```
interpreter  compiler/ObjClass.a24  FindOverload
compiler     bootstrap/algol.c      alg_invoke
conformance 0050-overload-selection.a24
```

[EXP-014] An overload is chosen by preferring an **exact match** on every argument; failing that, one reachable by widening [VAR-004], since a parameter is an assignment context [VAR-017]. When neither fits, the call fails with No matching signature for function.

So a `Char` argument selects a `Char` parameter where one is declared, and widens to a `String` parameter where it is not.

⚠ **PARTLY IMPLEMENTED.** Exact matching works and is correctly preferred — with both overloads declared, `Take('a')` selects the `Char`. Widening does not: with only a `String` overload declared, `Take('a')` is No matching signature for function. See DEF-10.

```
interpreter  compiler/ObjClass.a24  FindOverload
conformance 0050-overload-selection.a24
defect       DEF-10b-widening-at-a-parameter.a24
```

9.6 Subscripting

[EXP-015] Subscripting a String yields the Char at that **character** position, counted from zero [SRC-004]. An index outside the value is Index N out of range 0..M.

⚠ **PARTLY IMPLEMENTED.** The position is counted in bytes, so a subscript into text outside ASCII yields part of a sequence rather than a character. See DEF-01.

```
interpreter  compiler/Interpreter.a24  VisitSubscript
compiler     bootstrap/algol.c         alg_subscript_get
conformance  0051-string-subscript.a24
```

[EXP-016] A class instance may not be subscripted — see [TYP-010], and Annex H, H-4.

```
interpreter  compiler/Interpreter.a24  VisitSubscript
conformance  0031-instance-is-not-subscriptable.a24
```

9.7 Assignment

[EXP-017] Assignment is an **expression**, and its value is the value assigned: `X := (Y := 1)` leaves both at 1.

```
interpreter  compiler/Parser.a24  Assignment
unit         Resolve Assignment
conformance  0052-assignment-is-an-expression.a24
```

10. Statements

10.1 Blocks and expression statements

[STM-001] A block is `begin ... end` and may be empty.

```
interpreter  compiler/Interpreter.a24  VisitBlockStmt
unit         Execute Block Statement
conformance  0053-blocks-and-conditionals.a24
```

[STM-002] A declaration may **not** stand as the body of a branch or a loop. `if C then var X := 1;` is refused; the declaration must be inside a block.

⚠ **NOT YET IMPLEMENTED.** The interpreter accepts it, and the declared name **escapes into the enclosing scope** — so whether the name exists is decided by a runtime condition:

```
if True then var X := 1;   WriteLn (X);    // 1
if False then var X := 1;  WriteLn (X);    // Undefined variable 'X'.
```

A loop body that is never entered behaves the same way. See DEF-17.

⚠ The C back end already refuses this (C-12), so fixing the defect **removes** a divergence rather than creating one — the only rule in this specification so far of which that is true. It is also why the construct has no defender: either the declaration is dead, or the name leaks conditionally.

```
interpreter  compiler/Parser.a24  Statement
defect       DEF-17-declaration-as-an-unbraced-body.a24
```

10.2 Conditionals

[STM-003] `if Cond then S` with an optional `else S`. The condition is tested for truthiness [VAL-008], not required to be a Boolean. A missing `then` is Expect 'then' after if condition.

```
interpreter  compiler/Interpreter.a24  VisitIfStmt
unit        Execute If Statement
unit        Execute Else Statement
unit        Parse If Expect Then
conformance  0053-blocks-and-conditionals.a24
```

[STM-004] An `else` binds to the **nearest** unmatched `if`.

```
interpreter  compiler/Parser.a24  IfStatement
conformance  0053-blocks-and-conditionals.a24
```

10.3 Loops

[STM-005] `while Cond do S`. A missing `do` is Expect 'do' after condition.

```
interpreter  compiler/Interpreter.a24  VisitWhileStmt
unit        Execute While Loop
unit        Parse While Expect Do
conformance  0054-loops.a24
```

[STM-006] The counted form is `for Init ; Cond ; Step do S`, and it **desugars into a block** holding the initializer and a `while` — which is why its variable is scoped [DCL-008].

```
interpreter  compiler/Parser.a24  ForStatement
unit         Execute For Loop
unit         Parse For Statement
conformance  0054-loops.a24
```

[STM-007] `for var X in C do S` walks a collection or a String. Over a String it yields each `Char`; over a Map it yields each **key**.

```
interpreter  compiler/Interpreter.a24  VisitForInStmt
conformance  0054-loops.a24
```

[STM-008] Iterating anything else is Can only iterate a collection or a String. — see [TYP-011], and Annex H, H-5.

```
interpreter  compiler/Interpreter.a24  VisitForInStmt
conformance  0032-instance-is-not-iterable.a24
```

[STM-009] ⚠ The collection is **snapshotted** when the loop begins. Adding to it inside the loop does not lengthen the walk.

```
interpreter  compiler/Interpreter.a24  VisitForInStmt
compiler     bootstrap/algol.c         alg_iterable
conformance  0055-loop-snapshot.a24
```

[STM-010] `break` leaves the innermost enclosing loop. Outside a loop it is refused **where it is written** — a parse-time check — with `Must be inside a loop to use 'break'`.

```
interpreter  compiler/Parser.a24  BreakStatement
unit         Parse Break Inside A While
unit         Parse Break Outside A Loop
conformance  0054-loops.a24
refusal      0021-break-outside-a-loop.a24
```

10.4 Case

[STM-011] A `case` names a value, then arms, then `end`. An arm may list several values separated by commas, and a final `else` is optional.

```
CaseStmt = "case" Expression "of" { Arm } [ "else" Statement ] "end" .
Arm       = Expression { "," Expression } ":" Statement .
```

```
interpreter  compiler/Parser.a24  CaseStatement
conformance 0056-case.a24
```

[STM-012] ⚠ `case` **desugars into an if/else-if chain**. There is no case statement downstream of the parser, and two consequences follow from that rather than from any rule of their own:

- Arms compare with `=` [VAL-009], so a `Double` matches an `Integer` arm of the same value — `1.0` matches the arm `1`.
- There is no fall-through. At most one arm runs.

```
interpreter compiler/Parser.a24 CaseStatement conformance 0056-case.a24
```

[STM-013] When no arm matches and there is no `else`, nothing happens and execution continues after the `end`.

```
interpreter  compiler/Parser.a24  CaseStatement
conformance 0056-case.a24
```

10.5 Exit

[STM-014] `Exit` returns from the enclosing function or procedure, with a value in a function and bare in a procedure. Statements after it do not run.

```
interpreter  compiler/Interpreter.a24  VisitReturnStmt
conformance 0057-exit.a24
```

[STM-015] `Exit` at the top level is refused with `Can't return from top-level code`.

```
interpreter  compiler/Resolver.a24  VisitReturnStmt
unit         Invalid Return
refusal      0022-exit-at-top-level.a24
```

10.6 Exceptions

[STM-016] `raise E` carries **any value** — a `String`, an `Integer`, a class instance, anything.

```
interpreter  compiler/Interpreter.a24  VisitRaiseStmt
unit         Parse Raise
conformance 0058-exceptions.a24
```

[STM-017] A handler is written `on e : T do S` and matches on the runtime type name of the raised value. A handler for a base class catches a derived value.

```
interpreter  compiler/Interpreter.a24  VisitTryStmt
unit         Parse Try With A Typed Handler
conformance  0058-exceptions.a24
```

[STM-018] ⚠ The **most derived** matching handler runs, however the handlers are ordered. Writing the base first does not shadow the derived one.

⚠ This is a deliberate departure from first-match, and [STM-023] is what makes it total: two handlers for one type are refused, so "most derived" always names exactly one handler and there is never a tie to break by position. The commonest bug in a first-match language — a base handler written above a derived one, quietly swallowing everything — cannot be written here.

```
interpreter  compiler/Interpreter.a24  VisitTryStmt
compiler     bootstrap/algol.c         alg_handler
conformance  0058-exceptions.a24
```

[STM-023] Two handlers for the same type on one `try` are refused with `Duplicate handler for 'T'.`

```
interpreter  compiler/Interpreter.a24  VisitTryStmt
refusal      0023-duplicate-handler.a24
```

[STM-019] A handler written without `on` is the catch-all and matches any raised value.

```
interpreter  compiler/Interpreter.a24  VisitTryStmt
unit         Parse Untyped Handler Is The Catch All
unit         Parse Empty Except Is The Catch All
conformance  0058-exceptions.a24
```

[STM-020] A runtime error raised by the language is catchable **as a String**, carrying the diagnostic as its value: dividing by zero inside a `try` is caught by `on e : String with e equal to Division by zero.`

```
interpreter  compiler/Interpreter.a24  VisitTryStmt
compiler     bootstrap/algol.c         alg_error
conformance  0058-exceptions.a24
```

[STM-021] A value raised and never caught ends the program, printing `Uncaught:` followed by the value, and exits with status **70**.

```
interpreter  compiler/Main.a24  Main
compiler     bootstrap/algol.c  alg_raise
conformance  0059-uncaught-exits-70.a24
```

10.7 print

[STM-022] There is no print statement. `WriteLn` [RT-015] writes a value and a newline, and it is an ordinary built-in rather than syntax.

⚠ **NOT YET IMPLEMENTED.** The implementation has a `print E` statement, and `print` is a keyword there [LEX-010], so the word cannot be used as a name. See DEF-04.

```
interpreter  compiler/Interpreter.a24  VisitPrintStmt
unit         Execute Print Statement
defect       DEF-04-print-is-a-keyword.a24
```

This rule is stated in chapter 10 rather than being deleted, because a rule ID is permanent: [STM-022] has been cited, and a reader who follows the citation should find out what became of the statement rather than nothing.

11. Functions, procedures and closures

11.1 Declarations

[FUN-001] A subprogram is declared `function` or `procedure`. Parameters may be typed or untyped, and a return type is optional.

```
FunDecl = ( "function" | "procedure" ) identifier "(" [ Params ] ")"
          [ ":" Type ] ";" [ Sections ] Block .
Params  = identifier [ ":" Type ] { "," identifier [ ":" Type ] } .
```

```
interpreter  compiler/Parser.a24  ParseFunction
unit         Parse Function
unit         Parse Function No Open Parenthesis
conformance  0060-subprogram-declarations.a24
```

[FUN-002] A subprogram that returns without a value yields `nil`.

```
interpreter  compiler/Interpreter.a24  VisitReturnStmt
conformance  0060-subprogram-declarations.a24
```

[FUN-003] A **procedure** may not `Exit` a value. `Exit E;` inside one is refused; `Exit;` and falling off the end are the ways a procedure returns, and its result is always `nil` [FUN-002].

A **function** may `Exit` a value or not, and yields `nil` when it does not.

⚠ **NOT YET IMPLEMENTED.** The two words are interchangeable: a procedure may `Exit` a value and a caller may use it, so `procedure P(); begin Exit 7; end` followed by `WriteLn(P())` prints 7. See DEF-18.

⚠ The restriction is what makes the word mean something. Without it `procedure` is a comment, and a reader cannot tell from a declaration whether a call has a result worth using — which the C back end must also decide, and which every caller must otherwise guard.

```
interpreter  compiler/Parser.a24  ParseFunction
defect      DEF-18-a-procedure-may-exit-a-value.a24
```

[FUN-004] A declaration may not have more than 255 parameters: Can't have more than 255 parameters.

```
interpreter  compiler/Parser.a24  ParseFunction
unit        Parse Function More Than 255 Parameters
refusal     0024-too-many-parameters.a24
```

11.2 Parameters and results

[FUN-005] A call checks the number of arguments — see [EXP-011].

```
interpreter  compiler/Interpreter.a24  VisitCall
conformance  0049-call-failures.a24
```

[FUN-006] A subprogram's declared parameter types are enforced on every call, whether it is a top-level subprogram or a method. A parameter is an assignment context [VAR-017], so an argument must have the declared type, widen to it [VAR-004], or be `nil` [VAR-005].

⚠ **NOT YET IMPLEMENTED for a top-level subprogram.** `function F(N : Integer)` accepts a `String`, a `Double` or a `Boolean` without complaint. Only the arity is checked. A **method's** types are enforced [FUN-007], so the same annotation is a contract in one place and decoration in the other. See DEF-19.

```
interpreter  compiler/Interpreter.a24  VisitCall
defect      DEF-19-top-level-parameter-types-unenforced.a24
```

[FUN-007] A **method's** parameter types **are** enforced, because a method goes through overload selection [EXP-013]. Passing a `String` where `Integer` is declared is `No matching signature for function.`

```
interpreter  compiler/ObjClass.a24  FindOverload
refusal     0025-method-parameter-type-is-enforced.a24
```

A method's types are checked because overload selection compares whole signatures [EXP-013]. A top-level subprogram does not overload, so nothing ever compares its parameters — which is the cause of DEF-19 rather than a decision anyone made.

[FUN-008] A declared **return** type **is** enforced. Exit of a value that does not fit is Type mismatch!

```
interpreter  compiler/TypeChecker.a24  Assignable
refusal      0026-return-type-is-enforced.a24
```

11.3 Closures

[FUN-009] A nested subprogram captures the enclosing variables **by reference**, and the capture outlives the call that created it. A function returning a nested one hands back something that keeps reading and writing the same variable.

```
interpreter  compiler/ObjFunction.a24  ObjFunction
compiler     bootstrap/algol.c         alg_cell
conformance  0061-closures.a24
```

[FUN-010] Each call to the enclosing subprogram creates a **fresh** set of captured variables. Two counters made the same way do not share a count.

```
interpreter  compiler/Environment.a24  Environment
compiler     bootstrap/algol.c         alg_closure
conformance  0061-closures.a24
```

11.4 Subprograms as values

[FUN-011] A subprogram's name used without a call is a value. It may be assigned to a variable, passed as an argument, stored in a collection, and called from wherever it comes to rest.

```
interpreter  compiler/ObjFunction.a24  ObjFunction
compiler     bootstrap/algol.c         alg_call
unit         Interpret Local Function
conformance  0062-subprograms-as-values.a24
```

11.5 Nesting

[FUN-012] Subprograms may be declared inside subprograms, to any depth.

⚠ The C back end refuses more than one level (C-2), so `conformance/0063` is one of the cases expected to fail its compiled half.

```
interpreter  compiler/Parser.a24  ParseFunction
conformance 0063-nesting.a24
```

12. Classes and objects

12.1 Declaration

[CLS-001] A class is declared with an optional superclass, an optional header of sections, and a body.

```
ClassDecl = "class" identifier [ "(" identifier ")" ] ";"
           [ Sections ] "begin" { Member } "end" .
```

```
interpreter  compiler/Parser.a24  ClassDeclaration
unit        Parse Class Declaration
unit        Parse Class No Begin
conformance 0064-class-declaration-and-fields.a24
```

[CLS-002] Fields are declared in `var` sections of the header [VAR-009]; methods in the body. Visibility markers apply to both [DCL-011].

```
interpreter  compiler/Parser.a24  ClassDeclaration
conformance 0064-class-declaration-and-fields.a24
```

12.2 Construction

[CLS-003] An instance is made by **calling the class**: `Point(3, 4)` .

```
interpreter  compiler/Interpreter.a24  VisitCall
compiler     bootstrap/algol.c         alg_new
conformance 0065-construction.a24
```

[CLS-004] A constructor is a member named `constructor Init` . Construction checks its arity, and a class with no constructor takes no arguments — `C(1, 2)` on such a class is Expected 0 arguments but got 2.

```
interpreter  compiler/ObjClass.a24  FindMethod
conformance 0065-construction.a24
```

[CLS-005] ⚠ A field's initializer is evaluated **once per instance**, at construction. Two instances of a class whose field is `List := []` hold two different Lists.

```
interpreter  compiler/ObjClass.a24  SeedFields
compiler     bootstrap/algol.c      alg_class_field
conformance  0064-class-declaration-and-fields.a24
```

[CLS-006] A field with no initializer begins as `nil` [VAR-002].

```
interpreter  compiler/ObjClass.a24  SeedFields
conformance  0064-class-declaration-and-fields.a24
```

12.3 Members

[CLS-007] A field is read without parentheses and a method is called with them. There is no getter declaration — see [TYP-012].

```
interpreter  compiler/ObjInstance.a24  Get
unit         Call Setters And Getters
conformance  0066-members.a24
```

[CLS-008] Every instance answers `ClassName` , and does so **ahead of its fields**: the name belongs to the language, so a field of that name cannot take it.

```
interpreter  compiler/ObjInstance.a24  Get
compiler     bootstrap/algol.c        alg_property
conformance  0066-members.a24
```

[CLS-009] A class declaring `ToString()` decides how its instances render through `Str` [RT-006] and wherever a value is written [RT-015]. With none, an instance renders as its class name followed by `instance` — `C instance` .

```
interpreter  compiler/Interpreter.a24  Stringify
conformance  0066-members.a24
```

[CLS-010] Reading or calling a member the class does not have is `Undefined` property `'X'`.

```
interpreter  compiler/ObjInstance.a24  Get
unit        Call Undefined Getter
conformance 0067-undefined-property.a24
```

12.4 Inheritance

[CLS-011] `class D (B)` makes `D` inherit `B`'s fields and methods. A method of the same name overrides the inherited one.

```
interpreter  compiler/ObjClass.a24  FindMethod
conformance 0068-inheritance.a24
```

[CLS-012] `super.M()` calls the version above the class that declared the calling method, not above the runtime class.

⚠ It binds to the class that **declared** the method, not to the runtime class of `this`. A method in `B` calling `super.Who()` reaches `A`'s version even when the receiver is a `C` below `B`.

```
interpreter  compiler/Interpreter.a24  VisitSuperExpr
compiler     bootstrap/algol.c         alg_invoke_from
conformance 0068-inheritance.a24
```

[CLS-013] A class may not inherit from itself: A class can't inherit from itself.

```
interpreter  compiler/Resolver.a24  VisitClassStmt
unit        Inherit From Self
refusal     0027-inherit-from-self.a24
```

[CLS-014] A superclass must be a class, and is checked **where it is declared** rather than where an instance is later built. Naming something that is not a class is `'X' is not a class.`, beside the existing `A` class can't inherit from itself. [CLS-013].

Naming a name that denotes nothing is `Undefined variable 'X'.`

⚠ **PARTLY IMPLEMENTED.** The **timing is right** — the check fires at the declaration, and a program that never constructs the class is still refused — but the **message is wrong**: Only instances have properties. `.`, a sentence naming neither the class, nor the superclass, nor inheritance, and describing a property access the program never wrote. See DEF-20.

```
interpreter  compiler/Interpreter.a24  VisitClassStmt
unit        Inherit Not A Class
defect      DEF-20-superclass-message-is-wrong.a24
```

12.5 Objects

[CLS-015] object N; declares a class having exactly one instance, reached by the name itself. The instance is built on first use, so an object may refer to another declared later in the file.

```
interpreter  compiler/Parser.a24  ClassDeclaration
compiler    bootstrap/algol.c    alg_singleton
unit        An Object Takes Visibility Sections
conformance 0069-objects.a24
```

[CLS-016] An object is not callable. Config() is Can only call functions and classes.

```
interpreter  compiler/Interpreter.a24  VisitCall
conformance 0070-object-is-not-callable.a24
```

13. Enumerations

13.1 Declaration

[ENU-001] An enumeration is declared with type , naming its members in order.

```
EnumDecl = "type" identifier "=" "(" identifier { "," identifier } ")" ";" .
```

```
interpreter  compiler/Parser.a24  EnumDeclaration
conformance 0071-enumerations.a24
```

[ENU-002] Each member is bound as a **bare name** in the enclosing scope and is also reachable qualified as Type.Member . Both spellings denote the **same interned object**, so RED = Colour.RED is true.

```
interpreter  compiler/ObjEnum.a24  ObjEnumType
compiler    bootstrap/algol.c    alg_enum_member
unit        Both Spellings Of An Enum Member Type As The Enum
conformance 0071-enumerations.a24
```

[ENU-003] Two enumerations **may** share a member name. Declaring type First = (A, B); and type Second = (A, C); in one scope is accepted, and neither declaration is affected by the other.

⚠ **NOT YET IMPLEMENTED.** The second declaration is refused with 'A' is already defined. , so adding a member to one enumeration can break an unrelated one elsewhere in the program. See DEF-22.

```
interpreter  compiler/Interpreter.a24  VisitEnumStmt
defect      DEF-22-shared-enum-member-names.a24
```

[ENU-011] A **bare** member name bound by more than one enumeration in scope is ambiguous, and using it is refused with 'A' is ambiguous: First or Second. The qualified form [ENU-002] resolves it: First.A and Second.A are two different members.

A bare name bound by only one enumeration in scope is unambiguous and needs no qualifier, which is the ordinary case and the reason members bind bare at all.

⚠ **The refusal belongs to the use, not to the declaration.** Two enumerations that never meet an ambiguous use coexist without complaint, and a program is told about a name only where it actually cannot be resolved.

⚠ **NOT YET IMPLEMENTED.** The declaration is refused first [ENU-003], so no program reaches the ambiguous use. See DEF-22.

```
interpreter  compiler/Interpreter.a24  VisitEnumStmt
defect      DEF-22-shared-enum-member-names.a24
```

[ENU-004] Naming a member the type does not have is Undefined enum member 'X'.

```
interpreter  compiler/ObjEnum.a24  ObjEnumType
conformance  0072-unknown-enum-member.a24
```

13.2 Values

[ENU-005] Members compare by **identity** [VAL-011]. A member of one enumeration is never equal to a member of another, whatever they are called.

```
interpreter  compiler/Interpreter.a24  IsEqual
unit        An Enum Member Does Not Satisfy Another Enum
conformance  0071-enumerations.a24
```

[ENU-006] M is T is true for the member's own type and false for every other.

```
interpreter  compiler/Interpreter.a24  VisitIsExpr
unit        An Enum Type Name Types As Itself
conformance  0071-enumerations.a24
```

[ENU-007] A member renders as its bare name: Str(RED) is RED .

```
interpreter  compiler/ObjEnum.a24  ToString
conformance  0071-enumerations.a24
```

[ENU-008] Members are **not ordered**. `RED < GREEN` is `Operands must be numbers.`

⚠ A program that needs an order compares ordinals [ENU-010]. The operators are left alone deliberately: `<` on two members would have to mean position, and position is exactly the property [ENU-009] shows to be a trap when it acts implicitly.

```
interpreter  compiler/Interpreter.a24  VisitBinary
conformance  0073-enum-members-are-not-ordered.a24
```

13.3 The ordinal

[ENU-009] The **first member of every enumeration is falsey**, and every later member is truthy, because truthiness reads the member's position [VAL-008].

This is deliberate. It lets a program declare its own two-valued types and use them directly in a condition, with no comparison and no conversion:

```
type Flag    = (Off, On);
type Answer = (No, Yes);

if F then ...
```

⚠ **The discipline it asks for: put the absent, off or zero member first.** The position is part of the declaration's meaning, so reordering members changes the truth of every condition written over them — in the same way, and for the same reason, that reordering a `case` 's arms changes which one runs. The position is readable [ENU-010], so nothing here is hidden.

⚠ This compiler's own enumerations already follow the convention: `FUN_NONE` and `CLASS_NONE` are the first members of `FunctionType` and `ClassType` .

```
interpreter  compiler/Interpreter.a24  IsTruthy
compiler     bootstrap/algol.c         alg_truthy
conformance  0074-enum-truthiness.a24
```

[ENU-010] A member answers `Ordinal` , its **zero-based position** in the declaration. `RED.Ordinal` is 0 and `BLUE.Ordinal` is 2. It answers no other property.

⚠ **NOT YET IMPLEMENTED.** A member is opaque: `RED.Ordinal` is `Only instances have properties.` , so the position that decides [ENU-009] cannot be read by a program at all — and a program can discover whether a member is falsey only by testing it for truth. See DEF-21.

⚠ The ordinal is what a program needs to order members [ENU-008], to index an array by one, or to write one out and read it back. It exists in both runtimes already; it is simply not published.

```
interpreter  compiler/ObjEnum.a24  ObjEnum
defect      DEF-21-enum-ordinal-is-not-reachable.a24
```

14. Collections

⚠ This chapter describes the collections as built-ins, which is what they are today and not what they are meant to remain. The intent is to move everything not genuinely pinned to the core out into a unit written in Algol-24 — see Annex E for which of the five are pinned and by what, and Annex H, H-9 for the work. What survives in the core is `Array`, which nothing in the language can express, and the literal forms `[...]` and `[:]`, which hand out the names `List` and `Map` before any unit can.

⚠ The rules below are therefore expected to leave this specification, and their conformance cases with them. That is not a failure of either: a rule that stops describing the *language* because its subject became a *library* has been retired, not falsified, and the cases that pinned it become the unit tests of the unit that replaces it. They are worth writing now precisely because they are the behavioural target that unit has to meet.

⚠ One rule in this chapter is **not** provisional in that way. [COL-007] specifies insertion order for every collection, including `Set` and `Map`, and any replacement must reproduce it — it was specified rather than left to the representation because both processors must agree, and a unit is a third implementation with the same obligation.

14.1 Construction

[COL-001] A bracketed list of values is a `List`, and `[]` is an empty one. A bracketed list of `key : value` pairs is a `Map`, and `[:]` is an empty one.

```
interpreter  compiler/Parser.a24  Primary
conformance  0075-collection-construction.a24
```

[COL-002] `List()`, `Set()`, `Stack()` and `Map()` construct empty collections, `Set(L)` builds a `Set` from a collection, and `Array(N)` an `Array` of `N` elements.

```
interpreter  compiler/Interpreter.a24  ArrayNative
compiler     bootstrap/algol.c          alg_array
conformance  0075-collection-construction.a24
```

14.2 Members

[COL-003] Every collection answers `Length` and `IsEmpty` as properties, and `Contains` as a method. Everything else is per kind:

	List	Set	Stack	Array	Map
<code>Length</code> <code>IsEmpty</code> <code>Contains</code>	•	•	•	•	•
<code>Get</code>	•			•	•
<code>Add</code>	•	•			
<code>Insert</code> <code>RemoveAt</code>	•				
<code>IndexOf</code> <code>Sort</code>	•			•	
<code>Clear</code>	•	•	•		•
<code>Remove</code>		•			•
<code>Set</code> <code>Fill</code>				•	
<code>Put</code> <code>Keys</code> <code>Values</code>					•
<code>ToList</code>		•			
<code>Push</code> <code>Pop</code> <code>Peek</code>			•		

⚠ This table is checked against the interpreter by `spec/spec.sh`, which asks `spec/members.a24` which members each kind actually answers for. A matrix transcribed into a specification and checked by nobody is the most rot-prone thing this document can hold.

```
interpreter  compiler/ObjCollection.a24  Get
conformance 0076-collection-members.a24
```

[COL-004] ⚠ A `List` has **no** `Remove`. Removing a value from a `List` means finding it with `IndexOf` and passing that to `RemoveAt`, while a `Set` and a `Map` remove by value and by key directly.

⚠ The asymmetry has a reason, and it is not an oversight. A `List` may hold the same value more than once, so "remove this value" has no single meaning — the first, the last, or all of them — while a `Set` holds each value once and a `Map` each key once, so for those it has exactly one. Making a program write `IndexOf` and `RemoveAt` is making it say which it meant.

```
interpreter  compiler/ObjCollection.a24  Get
conformance 0076-collection-members.a24
```

[COL-005] A member a kind does not have is `Undefined` property `'X'`.

```
interpreter  compiler/ObjCollection.a24  Get
conformance  0077-undefined-collection-member.a24
```

[COL-006] A collection member's name is matched **case-insensitively**, as every name in the language is [SRC-011]. `L.Add(2)` and `L.add(2)` are the same member.

⚠ **NOT YET IMPLEMENTED.** The interpreter matches exactly, so `L.add(2)` is Undefined property 'add'. See DEF-02.

⚠ **This is the one place where the C back end is already right and the interpreter is the one to change** (C-4). The compiler compares these names with `alg_stricmp`, which was recorded as a divergence when the interpreter was taken to be the authority; [SRC-011] reverses that. It is also the safer direction to move, since it can only make programs start working.

```
interpreter  compiler/ObjCollection.a24  Get
defect      DEF-02-identifiers-are-case-sensitive.a24
```

14.3 Order

[COL-007] Every collection iterates in **insertion order**, including `Set` and `Map`. This is specified rather than left to the representation, because both processors must produce the same output.

```
interpreter  compiler/ObjCollection.a24  Items
compiler     bootstrap/algol.c           alg_iterable
conformance  0078-collection-order.a24
```

[COL-008] Re-assigning an existing `Map` key keeps the key's **original position**. `Put` on a key already present replaces the value and does not move it to the end.

```
interpreter  compiler/ObjCollection.a24  Invoke
compiler     bootstrap/algol.c           alg_put
conformance  0078-collection-order.a24
```

[COL-009] `Keys()` and `Values()` answer in that same order, so the two correspond element for element.

```
interpreter  compiler/ObjCollection.a24  Invoke
conformance  0078-collection-order.a24
```

14.4 Behaviour

[COL-010] A `Set` holds each value once. Adding a value it already has leaves its length unchanged.

```
interpreter  compiler/ObjCollection.a24  Invoke
conformance  0079-collection-behaviour.a24
```

[COL-011] ⚠ Remove answers **different kinds of thing** by kind. A `Map` returns the value removed, and `nil` when the key was absent. A `Set` returns whether there was anything to remove.

⚠ Each answer is the useful one for its kind — a `Map`'s removed value is worth having, and a `Set` has nothing to hand back but whether it did anything — but the two cannot be used interchangeably, and nothing in the call says which is coming.

```
interpreter  compiler/ObjCollection.a24  Invoke
compiler     bootstrap/algol.c           alg_remove
conformance  0079-collection-behaviour.a24
```

[COL-012] Membership — `Contains`, `in`, and `Map` key lookup — uses the equality of [VAL-009], so a collection holding `1.0` contains `1`. See [VAL-013].

⚠ **NOT YET IMPLEMENTED.** All three compare strictly, without numeric promotion. See DEF-14.

```
interpreter  compiler/ObjCollection.a24  Invoke
compiler     bootstrap/algol.c           strict_equals
defect       DEF-14-membership-does-not-follow-equality.a24
```

[COL-013] `Sort` orders in place and is **stable**. It orders numbers against numbers and text against text; mixing them is Can only sort numbers against numbers, or text against text.

```
interpreter  compiler/ObjCollection.a24  Invoke
compiler     bootstrap/algol.c           alg_sort
conformance  0080-sort.a24
```

[COL-014] Subscripting reads by position for a `List` and an `Array` and by key for a `Map`. A `Set` has no positions and is refused with `Subscript` target should be an ordinal.

```
interpreter  compiler/ObjCollection.a24  At
compiler     bootstrap/algol.c           alg_subscript_get
conformance  0081-subscripting-by-kind.a24
```

[COL-015] An `Array` is fixed in size and does not grow on assignment — see [TYP-008].

```
interpreter  compiler/ObjCollection.a24  At
conformance  0029-array-is-fixed.a24
```

15. Modules

15.1 Importing

[MOD-001] `uses` imports another file. A bare identifier names the file of that name with `.a24` appended; a quoted string is a path.

```
UsesStmt = "uses" ( identifier | string_lit ) ";" .
```

```
interpreter  compiler/Parser.a24  UsesStatement
conformance  0082-module-import.a24
```

[MOD-002] ⚠ A module name is the one place [SRC-011] does **not** reach. It names a file, and the filesystem decides how that name is matched — case- insensitively on macOS and Windows, sensitively on Linux. `uses scanner` may therefore find `Scanner.a24` on one machine and fail on another.

This is stated rather than fixed because the language does not own the filesystem. A program that wants to run everywhere spells a module name exactly as the file is named.

A module is looked for **beside the importing file first**, then in the working directory. Two directories may therefore hold files of one name without either reaching the other's. Failure is `Could not find module 'X': no X.a24 in ...`

```
interpreter  compiler/Parser.a24  ResolveModule
conformance  0082-module-import.a24
```

[MOD-003] A module is loaded and executed **once**, keyed by its resolved path, however many files import it. A second import of the same file sees the names without re-running the body.

```
interpreter  compiler/Parser.a24  UsesStatement
conformance  0083-module-runs-once.a24
```

[MOD-004] A file may open with `unit N; .` If present, `N` must match the file's own name: `Unit 'Wrong'` must match its file name `'Mismatch'`.

```
interpreter  compiler/Parser.a24  UnitHeader
refusal      0029-unit-name-must-match-the-file.a24
```

15.2 Exports

[MOD-005] A module exports its top-level declarations, except those marked `private` .

```
interpreter  compiler/Interpreter.a24  VisitModuleStmt
conformance 0082-module-import.a24
```

[MOD-006] At the top level of a module, `private` precedes a **single declaration** and hides it. It is not a section marker there, unlike inside a class [DCL-011].

```
interpreter  compiler/Parser.a24  RecordPrivate
unit         Module Private Is Not A Section Marker
conformance 0082-module-import.a24
```

[MOD-007] A private name is invisible to an importer both bare and qualified. Qualified, it is `Undefined` name `'Hidden'` in unit `'Mid'`.

```
interpreter  compiler/Interpreter.a24  Qualified
conformance 0084-module-private.a24
```

[MOD-008] Two imported modules **may** export one name. Importing both is accepted, and neither module is affected by the other.

 **NOT YET IMPLEMENTED.** The second import is refused with `'Clash'` is already defined; mark it `private` in one of the modules. — so a module cannot be imported alongside another that happens to export a name it also exports, and the advice given is to change one of them. See DEF-23.

```
interpreter  compiler/Interpreter.a24  VisitModuleStmt
defect       DEF-23-modules-may-not-share-exported-names.a24
```

[MOD-013] A **bare** name exported by more than one imported module is ambiguous, and using it is refused with `'Shared'` is ambiguous: `Alpha` or `Beta`. The qualifier [MOD-010] resolves it: `Alpha.Shared()` and `Beta.Shared()` are two different functions.

A bare name exported by only one imported module is unambiguous and needs no qualifier, which is the ordinary case.

 **The refusal belongs to the use, not to the import.** Two modules that export a common name and are never used ambiguously coexist without complaint, and a program is told about a name only where it actually cannot be resolved.

 This is [ENU-011] applied to units, and for the same reason: a name that cannot be resolved is a property of the *use*, not of the declaration. It also removes the need for the advice the old diagnostic gave — a module should not have to be edited because of what some other module, possibly written by someone else, happens to export.

⚠ **NOT YET IMPLEMENTED.** The import is refused first [MOD-008], so no program reaches the ambiguous use. See DEF-23.

```
interpreter  compiler/Interpreter.a24  VisitModuleStmt
defect      DEF-23-modules-may-not-share-exported-names.a24
```

15.3 Visibility

[MOD-009] ⚠ uses is **not transitive**. If A imports B and B imports C, then A does not see C's names — B may use them, and A may not. The diagnostic names the unit that would export it:

```
Undefined variable 'DeepName'. Unit 'Deep' exports it; this file has no 'uses' for it.
```

```
interpreter  compiler/Interpreter.a24  LookupVariable
conformance  0085-uses-is-not-transitive.a24
```

[MOD-010] An exported name may be qualified by its unit — Mid.MidName() — and the qualifier is resolved statically as a unit rather than evaluated as a value.

```
interpreter  compiler/Interpreter.a24  Qualified
conformance  0082-module-import.a24
```

[MOD-011] System is the unit of the built-in functions. No file imports it and every file may qualify against it: System.Copy('abcdef', 0, 3) .

```
interpreter  compiler/Resolver.a24  Units
conformance  0086-system-unit.a24
```

15.4 Cycles

[MOD-012] A cycle **between modules** works. A uses B and B uses A, and both are loaded, both bodies run, and the functions of each are callable — because a module is loaded once by resolved path [MOD-003], so the second import finds the entry already made rather than descending again.

Cycles of three and more behave the same way.

```
interpreter  compiler/Parser.a24  UsesStatement
conformance  0087-cycles-between-modules-work.a24
```

[MOD-014] A cycle **through the root file** — a module importing the file that is being run — does **not** work. The root's own imported names are undefined when its body runs: Undefined variable 'ModName'. , after

root body ran has already printed.

⚠ **NOT YET IMPLEMENTED.** The specification requires a root cycle to behave as [MOD-012] does, or to be refused by name. It does neither. See DEF-24.

⚠ The cause is that the root is never entered in the loader's map of loaded files, so a module importing it back parses it a **second** time and the two copies do not share their names. Compiled, the same shape refuses with Two modules named 'X' is not supported by the C back end yet. (C-1), which at least names the right subject.

```
interpreter  compiler/Parser.a24  UsesStatement
defect      DEF-24-a-cycle-through-the-root.a24
```

16. Built-in functions

16.1 The set

[RT-001] Twenty-five names are built in. Twenty-two are always available:

Length	Copy	Pos	Str	Ord	Char	Val
Max	Mod	clock				
List	Set	Stack	Array	Map	Buffer	
TextFile		FileExists				
ParamCount		ParamStr				
Write	WriteLn					

⚠ The list is checked against the names the interpreter registers by spec/spec.sh ; Annex B is the index.

```
interpreter  compiler/Interpreter.a24  Builtins
conformance  0088-builtins.a24
```

[RT-002] The remaining three — AssertTrue , AssertEqual and Fail — are registered **only while** -- test **is running** [see 19]. Calling one outside a test run is Undefined variable 'AssertTrue'.

```
interpreter  compiler/Interpreter.a24  RunTests
refusal     0030-assert-outside-a-test-run.a24
```

16.2 Text

[RT-003] Length(V) measures **text**, in characters [SRC-004]. It takes a String or a Char. Given a collection it is refused — Length expects text; use .Length for a collection. — because a collection's count

is a property [COL-003] and the two are different questions.

⚠ **NOT YET IMPLEMENTED, in two ways.** It stringifies whatever it is given and measures that, so `Length([10, 20, 30])` is **12** — the length of the rendering — where `L.Length` is 3. And it measures bytes rather than characters. See DEF-25 and DEF-01.

⚠ **The failure mode is the bad one:** the wrong call returns a *plausible number* rather than an error, and for a List of one-digit numbers the two even coincide at small sizes before diverging. A program asking `Length(L)` gets an answer, uses it, and is wrong.

⚠ A program that means the rendering writes `Length(Str(L))`, which is what it is getting today by accident.

interpreter	compiler/Interpreter.a24	LengthNative
compiler	bootstrap/algol.c	alg_length
defect	DEF-25-length-of-a-collection.a24	

[RT-017] A `String` answers `Length` as a **property**, its count of characters: `'abc'.Length` is 3. This is the same count `Length('abc')` gives, and the same spelling every collection uses [COL-003].

⚠ **NOT YET IMPLEMENTED.** The interpreter refuses it with `Only instances have properties`. The **compiled** back end already answers 3, so this is one of the two rules in this specification where the C back end is right and the interpreter is the one to change (C-9, with [COL-006] the other). See DEF-26.

⚠ A `String` is already iterable [STM-007] and subscriptable [EXP-015]; not answering for its own length was the odd one out.

interpreter	compiler/ObjInstance.a24	Get
compiler	bootstrap/algol.c	alg_property
defect	DEF-26-a-string-has-no-length-property.a24	

[RT-004] `Copy(Text, Begin, Length)` takes a substring, counting from zero. The length is clamped to what remains, so `Copy('abcdef', 3, 99)` is `def`. A start outside the text is `Copy failed: Start -2 out of range 0..6`.

interpreter	compiler/Interpreter.a24	CopyNative
compiler	bootstrap/algol.c	alg_copy
conformance	0089-text-builtins.a24	

[RT-005] `Pos(Text, Part)` answers the zero-based index of `Part` within `Text`, or **-1** when it is absent.

interpreter	compiler/Interpreter.a24	PosNative
compiler	bootstrap/algol.c	alg_pos
conformance	0089-text-builtins.a24	

[RT-006] `Str(V)` renders any value: an `Integer` bare, a `Double` always with a point (`1.0`), a `Boolean` lowercase (`true`), `nil` as `nil` , a `List` as `[10, 20, 30]` , a `Map` as `[1:2]` , and an instance by its `ToString` [CLS-009].

```
interpreter  compiler/Interpreter.a24  Stringify
compiler    bootstrap/algol.c         alg_str
conformance 0090-str.a24
```

[RT-007] `Ord(C)` answers the code point of a single character, as an **Integer**. Anything longer is `Ord` failed: `'ab'` has no ordinal.

```
interpreter  compiler/Interpreter.a24  OrdNative
compiler    bootstrap/algol.c         alg_ord
conformance 0089-text-builtins.a24
```

[RT-008] `Char(N)` answers the character with code point `N` , over the range of [LEX-025] — `0 ... 10FFFF`, excluding the surrogates. `Ord` and `Char` are inverse across it.

⚠ **NOT YET IMPLEMENTED.** The range is `0 ... 127`. See DEF-06.

```
interpreter  compiler/Interpreter.a24  CharNative
compiler    bootstrap/algol.c         alg_char
defect      DEF-06-char-range-and-diagnostic.a24
```

16.3 Numeric

[RT-009] `Val(S)` parses a number from text, answering an **Integer** where the text has no point and a **Double** where it has one — reading the same characters the literal rules do [LEX-015], [LEX-020]. Failure is `Val` failed: `'abc'` is not a number.

⚠ **NOT YET IMPLEMENTED.** It always yields a `Double`, so `Val('42')` is `42.0` and `Val('42')` is `Integer` is false. See DEF-27.

```
interpreter  compiler/Interpreter.a24  ValNative
compiler    bootstrap/algol.c         alg_val
defect      DEF-27-val-and-max-do-not-meet.a24
```

[RT-010] `Max(A, B)` takes any two numbers and promotes as every other numeric operator does [EXP-005], so `Max(3.5, 2)` is `3.5` .

⚠ **NOT YET IMPLEMENTED.** It takes `Integers` only: `Max(3.5, 2)` is `Max` expects `Integers`. See DEF-27.

⚠️ **[RT-009] and [RT-010] are one defect, not two.** Individually each is defensible; together they leave `Max(Val(A), Val(B))` failing for **every** input, so text holding two numbers cannot be compared without going outside both built-ins.

```
interpreter  compiler/Interpreter.a24  MaxNative
compiler    bootstrap/algol.c         alg_max
defect      DEF-27-val-and-max-do-not-meet.a24
```

[RT-011] `Mod(A, B)` answers the remainder, whose sign follows the dividend: `Mod(-7, 3)` is `-1`. A zero divisor is `Mod` failed: Division by zero.

```
interpreter  compiler/Interpreter.a24  ModNative
compiler    bootstrap/algol.c         alg_mod
conformance  0091-numeric-builtins.a24
```

[RT-012] `clock()` answers the seconds since the epoch as a **Double**, at millisecond resolution.

```
interpreter  compiler/Interpreter.a24  ClockNative
compiler    bootstrap/algol.c         alg_clock
conformance  0091-numeric-builtins.a24
```

16.4 Environment

[RT-013] `ParamStr(0)` is the program's own name and `ParamCount()` does not count it, so a program run with no arguments reports zero.

```
interpreter  compiler/Interpreter.a24  ParamCountNative
compiler    bootstrap/algol.c         alg_param_count
conformance  0092-environment-builtins.a24
```

[RT-014] `FileExists(Name)` answers whether the named file exists.

```
interpreter  compiler/Interpreter.a24  FileExistsNative
compiler    bootstrap/algol.c         alg_file_exists
conformance  0092-environment-builtins.a24
```

[RT-015] `Write(V)` and `WriteLn(V)` write the stringified value to standard output, `WriteLn` following it with `#10` — always that byte, never the host's line separator, so one program writes the same bytes everywhere.

```
interpreter  compiler/Interpreter.a24  WriteLnNative
compiler    bootstrap/algol.c         alg_writeln
conformance  0093-write-and-writeln.a24
```

[RT-016] `ReadLn` on a `TextFile` splits on the same rule as the scanner [SRC-006], [SRC-007]: a line ends at `#10` , which is **not** returned; a `#13` immediately before it comes off with it; and any other `#13` is ordinary text returned as part of the line.

A file whose only line endings are `#13` is therefore read as a single line containing those bytes.

```
interpreter  compiler/ObjFile.a24  Invoke
compiler     bootstrap/algol.c     file_read_line
conformance  0008-readln-line-rule.a24
```

17. Program initialization and execution

17.1 A program

[INI-001] A program is one file. Its top-level statements are executed in the order they are written, and there is no distinguished entry point — no `main` , and no statement that begins execution.

⚠ The compiled back end does not preserve this order for a bare top-level block (C-11) or for module bodies (C-5), so `conformance/0094` is expected to fail its compiled half.

```
interpreter  compiler/Main.a24  Run
conformance  0094-program-order.a24
```

[INI-002] A **variable or constant** takes effect when its statement is reached, so a name is undefined above its declaration [DCL-016]. A **function or class** is visible throughout the file wherever it is written [DCL-006].

⚠ **NOT YET IMPLEMENTED for functions and classes.** The interpreter binds every top-level name when its declaration runs, so a call above a function is `Undefined variable 'F'`. See DEF-15.

```
interpreter  compiler/Interpreter.a24  Interpret
defect       DEF-15-declarations-are-not-hoisted.a24
```

17.2 Module initialization

[INI-003] A `uses` loads and runs its module **at the point it appears**. Root statements written between two `uses` clauses therefore run between the two module bodies:

```
WriteLn ('1 root');      → 1 root
uses Alpha;              → Alpha body
WriteLn ('2 root');      → 2 root
```

```
uses Gamma;           → Gamma body
WriteLn ('3 root');    → 3 root
```

```
interpreter  compiler/Interpreter.a24  VisitModuleStmt
conformance  0095-module-init-order.a24
```

[INI-004] A module is initialized once [MOD-003], and its imports are initialized before it, so a module's own body may use anything it imported.

```
interpreter  compiler/Interpreter.a24  VisitModuleStmt
conformance  0095-module-init-order.a24
```

⚠ compile-only divergence, and a silent one. The compiled program runs **every** module initializer before **any** root statement, so the example above prints both module bodies first and then all three root lines. The same program produces two different orders. See Annex C, C-5.

17.3 Termination

[INI-005] A program that reaches the end of its statements exits with status **0**, and **only** such a program does. A run that never began — because the file could not be read — is a failure and exits non-zero [INI-006].

⚠ PARTLY IMPLEMENTED. A program that reaches the end does exit 0. A run that never began does **not** fail: `algc no-such-file.a24` prints `algc: cannot open no-such-file.a24` and exits **0**, so the driver reports a failure and reports success at the same time. Naming a directory prints nothing and also exits 0. See DEF-28.

⚠ DEF-28 has no case in `defects/`, **and cannot have one.** The fault is in the driver rather than in anything a program can do, and every case in the corpus is run by handing `algc` a file that exists. A case that ran and exited 0 would record exactly what a *correct* implementation produces, proving nothing — so the reproduction is a shell command, given in Annex F.

```
interpreter  compiler/Main.a24  Main
conformance  0094-program-order.a24
```

[INI-006] Every failure the language reports exits with status **70** — an uncaught `raise` [STM-021], and equally a scan, parse, resolution or type error, which are reported before any statement runs.

```
interpreter  compiler/Main.a24  CheckScanned
compiler     bootstrap/algol.c  alg_error
conformance  0096-exit-status.a24
```

⚠ **One status for every kind of failure** is deliberate rather than unconsidered. A caller wanting to tell a compile error from a runtime one reads the diagnostic; the alternative — a second status for failures found before execution — buys a little for tooling and costs every existing caller a change. What is *not* deliberate is a failure exiting **0**, which is DEF-28.

17.4 Arguments

[INI-007] A program reads its command line through `ParamCount` and `ParamStr` [RT-013].

`ParamStr(0)` is the program's own name, and arguments follow from index 1.

```
interpreter  compiler/Main.a24    ArgumentsFrom
compiler     bootstrap/algol.c    alg_set_arguments
conformance  0092-environment-builtins.a24
```

18. Errors

18.1 Phases

[ERR-001] A program passes through five phases: **scan**, **parse**, **resolve**, **check**, **run**. The first four complete over the whole program — its imports included — before any statement is executed.

```
interpreter  compiler/Main.a24  Run
conformance  0097-error-phases.a24
```

[ERR-002] An error in any of the first four phases means **no statement runs at all**. A program cannot produce output and then fail to compile.

```
interpreter  compiler/Main.a24  Run
conformance  0097-error-phases.a24
refusal      0008-declared-type-constrains.a24
```

[ERR-003] A runtime error occurs during execution. Statements before it have run and their output stands.

```
interpreter  compiler/Interpreter.a24  Interpret
conformance  0098-runtime-errors-follow-output.a24
```

18.2 Diagnostics

Diagnostics are part of the observable surface [1.2]; their wording and shape are specified.

[ERR-004] A **scan** error reads [line N] Error: <message> and carries no source excerpt. Like every error in the first four phases it prevents execution [ERR-002].

⚠ **That is the whole of the requirement.** This rule previously went on to say the error is "recorded rather than raised: the scanner sets a flag and keeps the message, and a driver must ask" — which describes how *this* implementation happens to work, not what an implementation must do. A specification that mandates a mechanism forbids a better one; what matters is that the error is reported in the shape above and that no statement runs. The mechanism, and the hazard that comes with it, are recorded in Annex G, G.1.

```
interpreter  compiler/Scanner.a24  HadError
unit         Scan Unrecognized Character Is Recorded
conformance  0099-scan-error-shape.a24
```

[ERR-005] A **parse** or **resolution** error prints the message and a three-line excerpt naming the file, the line, and the offending token:

```
Uncaught: Expect variable name.
[ERROR] e.a24: Expect variable name.
[ERROR] 2 | var := 1;
[ERROR]   | ^^^
```

```
interpreter  compiler/Console.a24  Error
conformance  0100-parse-error-shape.a24
```

[ERR-006] A **type** error carries the same three-line excerpt a parse error does [ERR-005], and names both types:

```
Uncaught: Expected Integer, found String.
[ERROR] e.a24: Expected Integer, found String.
[ERROR] 2 | var X : Integer := 'text';
[ERROR]   |           ^^^^^^
```

⚠ **NOT YET IMPLEMENTED.** It prints Uncaught: Type mismatch! and nothing else — no file, no line, no token, neither type, and the same five words for every mismatch the checker finds. See DEF-29.

⚠ **This compounds with [ERR-002]:** a type error stops the program before any statement runs, so there is no output to orient by either. The message is the only information available and it carries none.

```
interpreter  compiler/TypeChecker.a24  Assignable
defect       DEF-29-a-type-error-says-nothing.a24
```

18.3 Catching

[ERR-007] Only **runtime** errors are catchable, and they are caught as a String carrying the diagnostic [STM-020].

```
interpreter  compiler/Interpreter.a24  VisitTryStmt
conformance 0101-catching.a24
```

[ERR-008] A `try` around a scan, parse, resolution or type error catches nothing, because those phases complete before the `try` is reached. Wrapping a mistyped declaration in a handler does not suppress it.

```
interpreter  compiler/Main.a24  Run
conformance 0101-catching.a24
```

18.4 Status

[ERR-009] Every failure exits with status **70**, whichever phase reported it [INI-006].

⚠ A failure that never reaches a phase at all — a file that cannot be read — is still a failure and must not exit 0 [INI-005]. See DEF-28.

```
interpreter  compiler/Main.a24  Main
conformance 0096-exit-status.a24
```

19. Test blocks

A test block is a declaration that a test run executes and an ordinary run ignores. The report it produces is specified line for line, because it is the surface on which two implementations are compared.

19.1 Declaration

[TST-001] A test is written `test` followed by a **quoted literal** naming it, then a block.

```
TestDecl = "test" ( string_lit | char_lit ) ";" Block .
```

`test` is not a keyword [LEX-011]; it is recognised here by the quoted name that follows it, so a variable may still be called `test` .

⚠ **Either quoted form**, because the name is *text* and a one-character name is an ordinary thing to write. `'X'` is a Char rather than a String [LEX-023], and that distinction belongs to values rather than to a declaration naming itself.

⚠ **NOT YET IMPLEMENTED.** Only a String is accepted, so `test 'X';` is not recognised as a test declaration at all — it parses as the identifier `test` followed by a Char, and fails with `Expect ';' after expression.`, a diagnostic that names neither tests nor the length of the name. See DEF-31.

```
interpreter  compiler/Parser.a24  Declaration
conformance 0102-test-declaration.a24
defect       DEF-31-a-one-character-test-name.a24
```

[TST-002] A test block is a declaration and does not run when the program runs.

```
interpreter  compiler/Interpreter.a24  HoistTests
conformance  0102-test-declaration.a24
```

19.2 Running

[TST-003] A test run executes the test blocks **instead of** the program. The top-level statements do not run — only the declarations they would have created.

```
interpreter  compiler/Interpreter.a24  RunTests
conformance  0103-a-test-run.a24
```

[TST-004] Tests are collected from the root file and from every module it reaches, each file contributing once however many ways it is reached.

```
interpreter  compiler/Interpreter.a24  HoistTests
conformance  0104-test-collection-and-order.a24
```

[TST-005] Tests are reported **sorted by name within a file**, and files in the order their first test was met — which for `uses` is load order. Source order within a file is not preserved.

```
interpreter  compiler/Interpreter.a24  RunTests
conformance  0104-test-collection-and-order.a24
```

[TST-006] A program's own `Write` and `WriteLn` output is **swallowed** during a test run, so it cannot interleave with the report.

```
interpreter  compiler/Interpreter.a24  SuppressOutput
compiler     bootstrap/algol.c         alg_test_begin
conformance  0103-a-test-run.a24
```

[TST-007] A value raised inside a test body and not caught makes that test **fail**; it does not end the run, and later tests still execute.

```
interpreter  compiler/Interpreter.a24  RunTests
compiler     bootstrap/algol.c        alg_test_run
conformance  0105-report-format.a24
```

19.3 The report

[TST-008] The report consists of these lines, in this order:

```
[INFO] Running N tests...
[INFO] < file >
[INFO] Test: <name> <leader> [ PASS ]
[ERROR] <file>: <message>          only after a FAIL, and only interpreted
[INFO]
[INFO] All N tests passed.
```

A file's block is opened by its < file > line, and a blank [INFO] line separates files and precedes the summary.

```
interpreter  compiler/Interpreter.a24  Report
compiler     bootstrap/algol.c        alg_test_run
conformance  0105-report-format.a24
```

[TST-009] The dot leader is 55 - Length(name) dots, clamped to a minimum of one, so a name longer than the banner still produces a well-formed line.

```
interpreter  compiler/Interpreter.a24  Report
compiler     bootstrap/algol.c        alg_test_run
conformance  0106-dot-leader.a24
```

[TST-010] The report is coloured, and the colours are part of it: the [INFO] tag white and blue, [ERROR] white and red, the file name cyan, PASS green, FAIL red, and the summary green when all passed and red otherwise.

 The escapes are emitted **unconditionally**, whether or not the output is a terminal — the language has no way to ask — so anything reading a report strips or transliterates them.

```
interpreter  compiler/Console.a24  INFO
compiler     bootstrap/algol.c    INFO_TAG
conformance  0105-report-format.a24
```

[TST-011] The summary is All N tests passed. or N of M tests failed. , and the run exits **0** when every test passed and **70** when any failed.

```
interpreter  compiler/Interpreter.a24  RunTests
compiler     bootstrap/algol.c        alg_test_summary
conformance  0103-a-test-run.a24
```

19.4 Assertions

[TST-012] Three assertions exist, and only during a test run [RT-002]:

Call	Message on failure
AssertTrue(V)	Assertion failed. Expected true but got 'V'.
AssertEqual(E, A)	Assertion failed. Expected 'E' but got 'A'.
AssertEqual(E, A) , where the two render alike	Assertion failed. Expected T 'E' but got U 'A'.
Fail(M)	Failed. M

Two spaces follow the full stop in each.

⚠ The third form is not an alternative wording but a different case, and it is the reason the second is not enough: a Char and a String both render as 3 and are never equal [LEX-026], so a message quoting only the rendered values would read Expected '3' but got '3'. Naming the types is what makes that legible. Both processors already do this, and it was missing from this table.

⚠ NOT YET IMPLEMENTED. Every form begins Assertion 'left = right' failed. — including AssertTrue , which made no comparison and has no left or right. The stem reads like a template nobody filled in, and for AssertTrue it describes an equality test that did not happen while saying nothing about the value that was false. See DEF-30.

```
interpreter  compiler/Interpreter.a24  AssertTrueNative
compiler     bootstrap/algol.c        alg_assert_equal
defect       DEF-30-assertion-messages.a24
```

[TST-013] AssertEqual compares with = [VAL-009], so it promotes numerically and holds a Char unequal to a String [LEX-026].

```
interpreter  compiler/Interpreter.a24  AssertEqualNative
conformance  0107-assert-equal-comparison.a24
```

19.5 Compiled runs

[TST-014] The report is **the same from any implementation**, line for line and colour for colour. It is the surface on which two implementations are compared, so a difference in it is a difference in conformance and not a matter of presentation.

⚠ The interpreter meets this; the **compiled back end does not**. Its report omits the [ERROR] line after a failure, because compiled code carries no line information to build one from (C-3). Every other line is byte-identical, colour included. conformance/0105 pins the interpreted report, and its compiled half is expected to differ by exactly those lines — a compiler gap, not an interpreter defect.

⚠ This rule previously stated only that the compiled report differs, which said nothing about what an implementation must **do**. The requirement is agreement; C-3 is the current distance from it.

```
interpreter  compiler/Interpreter.a24  Report
compiler     bootstrap/algol.c        alg_test_run
conformance  0105-report-format.a24
```

Annex A — grammar summary (*non-normative*)

Every production stated in the chapters, collected. This annex adds nothing: each line appears in the chapter that specifies it, and spec/spec.sh checks that none has been added here or lost from here.

⚠ **The grammar is partial, deliberately.** Where a construct's shape was verified by running it rather than by writing a production, the chapter states it in prose and no production appears below. The gaps are named at the end of this annex rather than filled with plausible-looking rules, because a production nobody checked is exactly the kind of claim this specification exists to avoid.

Lexical

```
letter      = "a" ... "z" | "A" ... "Z" | "_" | unicode_letter .
decimal_digit = "0" ... "9" .
identifier_mark = "?" | "!" .

identifier   = letter { letter | decimal_digit | identifier_mark } .

integer_lit  = decimal_digit { decimal_digit } .
double_lit  = decimal_digit { decimal_digit } "." decimal_digit { decimal_digit } .
char_lit     = "'" source_character "'" | "#" decimal_digit { decimal_digit } .
string_lit   = '"' { source_character_other_than_quote | "'" } '"' .
```

Declarations

```

VarDecl    = "var" identifier [ ":" Type ] [ "!=" Expression ] ";" .
ConstDecl  = "const" identifier [ ":" Type ] "!=" Expression ";" .
Type       = identifier [ "of" identifier ] .

VarSection = "var" { identifier { "," identifier } [ ":" Type ]
               [ "!=" Expression ] ";" } .

FunDecl    = ( "function" | "procedure" ) identifier "(" [ Params ] ")"
               [ ":" Type ] ";" [ Sections ] Block .
Params     = identifier [ ":" Type ] { "," identifier [ ":" Type ] } .

ClassDecl  = "class" identifier [ "(" identifier ")" ] ";"
               [ Sections ] "begin" { Member } "end" .
EnumDecl   = "type" identifier "=" "(" identifier { "," identifier } ")" ";" .

UsesStmt   = "uses" ( identifier | string_lit ) ";" .
TestDecl   = "test" string_lit ";" Block .

```

Statements

```

CaseStmt = "case" Expression "of" { Arm } [ "else" Statement ] "end" .
Arm      = Expression { "," Expression } ":" Statement .

```

What is specified in prose instead

These constructs are specified by rule and verified by probe, without a production:

Construct	Specified by
Operator precedence and associativity	[EXP-001], [EXP-002] — a table, derived by running distinguishing cases
if , while , counted for , for ... in	[STM-003] ... [STM-007]
break , Exit , raise , print	[STM-010], [STM-014], [STM-016], [STM-022]
try / except / on handlers	[STM-017] ... [STM-019]
Blocks and expression statements	[STM-001], [STM-002]
Visibility sections	[DCL-011], [DCL-012]

⚠ Sections , Block , Member and Statement are referenced above and not themselves defined. Completing the grammar is worth doing; inventing those four productions from memory is not.

Annex B — index of built-in functions (*non-normative*)

The twenty-five built-in names, with the rule specifying each. `spec/spec.sh` checks this list against the names the interpreter actually registers.

Name	Rule	Summary
AssertEqual	[TST-012]	Fails unless two values are equal; test runs only
AssertTrue	[TST-012]	Fails unless a value is truthy; test runs only
Fail	[TST-012]	Fails outright with a message; test runs only
Array	[COL-002]	An Array of N elements, each <code>nil</code>
Buffer	[E.2]	Growable bytes with an explicit lifetime
List	[COL-002]	An empty List
Map	[COL-002]	An empty Map
Set	[COL-002]	An empty Set, or a Set of a collection's values
Stack	[COL-002]	An empty Stack
Char	[RT-008]	The character with a code point, 0 ... 127
Copy	[RT-004]	A substring, from a zero-based start, length clamped
Length	[RT-003]	⚠ The length of the argument's text , not a count
Ord	[RT-007]	The code point of one character, as an Integer
Pos	[RT-005]	A zero-based index, or -1 when absent
Str	[RT-006]	Any value rendered as text
Val	[RT-009]	⚠ A number parsed from text, always a Double
Max	[RT-010]	⚠ The greater of two Integers only
Mod	[RT-011]	The remainder, its sign following the dividend
clock	[RT-012]	Seconds since the epoch, as a Double
FileExists	[RT-014]	Whether a named file exists
TextFile	[E.2]	A text file handle
ParamCount	[RT-013]	The argument count, not counting the program
ParamStr	[RT-013]	An argument by index; 0 is the program

Name	Rule	Summary
Write	[RT-015]	Writes a value
WriteLn	[RT-015]	Writes a value and #10

⚠ `clock` is the only built-in spelled in lower case, and the only one whose name is not a noun or an imperative. Nothing depends on this; it is noted because a reader will wonder.

Annex C — compiler divergences (*non-normative*)

Where the C back end does not do what the interpreter does. The interpreter is the authority [1.1], so every entry here is a defect in the compiler rather than a choice the language has made.

⚠ A **loud** divergence refuses to compile and says why. A **silent** one produces a program that runs and behaves differently. The second kind is far worse, and the column says which each is.

⚠ **Nothing here is tracked by a suite of its own.** A divergence is not a third kind of case; it is an *outcome*. Every case in `conformance/` runs under both processors, and a case the interpreter gets right and the compiler does not fails its compiled half — that failure **is** the record of the divergence, and `conform.sh` reports it as a gap rather than a failure. The classification of a case never depends on the compiler's state:

The interpreter is	The case goes in
right	<code>conformance/</code> or <code>refusals/</code> , even if the compiler is wrong
wrong	<code>defects/</code> , even if the compiler is right

This follows the generation plan rather than tidiness. The goal of the next generation is an interpreter that matches this specification; the goal of the one after is a compiler that matches the interpreter. Compiler gaps are therefore **expected** to be red throughout the first of those, and the count is a progress measure for the second.

⚠ The one thing that must not break is the compiler's ability to **build and reproduce itself** — `./fixedpoint.sh` and `./test.sh` . A compiler that cannot compile cannot produce the generation that fixes these entries.

⚠ **An opt-out records nothing and notices nothing.** Twelve conformance cases once carried a `//` compiled: no marker to keep the suite quiet. C-14 was found within minutes of removing them, in a case that had been opted out since it was written — and C-9, C-11 and C-13 had no reproduction at all while the markers stood.

C-1 — A file in an import cycle with the root will not compile. (*loud*)

The root file is never entered in the parser's `Loaded` map, so a module importing the root back parses it a second time, and the emitter then sees two units of one name:

Two modules named 'Parser' is not supported by the C back end yet.

`compiler/Parser.a24` uses `Interpreter`, which uses `Parser`, so the compiler's own source is an instance. The same file runs correctly interpreted. This is currently the only known case of a valid program having no compiled form.

C-2 — Functions may not nest more than one level deep. *(loud) (refers to [FUN-012])*

A function nested more than one level deep is not supported by the C back end yet.

Three levels of nesting run correctly interpreted and refuse to compile.

C-3 — A compiled assertion failure carries no message. *(loud enough)*

`alg_test_run` prints no `[ERROR]` line, because compiled code has no line information to put in one. The `FAIL` stands alone where the interpreter also prints the assertion and a caret. A report comparison drops those lines for that reason.

C-4 — Collection member names are matched case-insensitively. *(silent) (refers to [COL-006])*

```
var L := [1];  
L.add (2);
```

Interpreted this is `Undefined property 'add'`. Compiled it works, and the program prints a length of 2.

⚠ This is the first **silent** divergence recorded, and it runs in the more dangerous direction: the compiler **accepts a program the language refuses**. A program developed against the compiler can use lowercase member names throughout and fail everywhere the moment it is run interpreted, with no warning from either processor that the two disagree.

The C runtime compares collection members with `alg_stricmp`, which is deliberate — but `bootstrap/algol.h` asserted alongside it that this was "what the interpreter does", and it is not. That comment has been corrected.

The interpreter is the authority [1.1], so the fault is the compiler's.

Fix: compare exactly in `alg_property` and `alg_invoke` for collection receivers. Note that `Length` and `IsEmpty` are matched case-insensitively by the same code, so the change must cover those too, and any program relying on the looser spelling will stop compiling — which is the point.

C-5 — Module bodies run at a different time. *(silent) (refers to [INI-003], [INI-004])*

Interpreted, `a` uses runs its module where it appears, so root statements interleave with module bodies in source order. Compiled, every module initializer runs before any root statement.

	interpreted	compiled
<code>WriteLn ('1 root');</code>	1 root	Alpha body
<code>uses Alpha;</code>	Alpha body	Gamma body
<code>WriteLn ('2 root');</code>	2 root	1 root
<code>uses Gamma;</code>	Gamma body	2 root
<code>WriteLn ('3 root');</code>	3 root	3 root

⚠️ Silent, and unlike C-4 it needs no unusual spelling to provoke: any program whose modules print, open a file, or set a variable the root then reads will behave differently under the two processors, and nothing warns.

The compiled shape follows from how the C is emitted — `main` calls each `init_<Unit>()` and then `init_Main()` — and it is the easier order to produce, since a module's initializer is a function and the root's body is not special.

The interpreter is the authority [1.1], so the compiler is wrong.

Fix: emit the root's top-level statements in place, interleaved with the `init_<Unit>()` calls that correspond to its `uses`, rather than collecting them all into `init_Main()` after the fact. That is a change to how the root unit is emitted and not to the runtime.

Not a divergence, and worth stating as a requirement: interpreted and compiled `--test` reports are byte-identical, colour included — 239 lines and 1,416 escape sequences for the full suite. Any difference between them is a defect in one or the other.

C-6 — Reading a method as a property crashes the compiled program. (*silent, and the worst kind*) (refers to [TYP-012])

```
class Box;
begin
  function Size (); begin Exit 7; end
end

var B := Box ();
WriteLn (B.Size ());
WriteLn (B.Size);
```

Interpreted this prints `7` and then `<fn Size>`, which is what [TYP-012] requires. Compiled it dies of SIGSEGV (exit 139) with **no output at all** — the earlier `WriteLn` is lost with the buffer.

⚠️ This is the most serious divergence recorded so far, and it is worse than C-4. C-4 accepts a program the language refuses; this one takes a program both processors accept and crashes it, without a diagnostic, in the

processor that is supposed to be the fast one. `alg_property` has no case for a method reached without a call.

⚠ It is also the reason `conformance/0033` carries `// compiled: no`. The case pins the interpreted behaviour, which is the language; the compiled half cannot run at all, and a harness cannot record a crash as an expectation.

C-7 — Two runtime diagnostics are worded differently. (*loud*) (refers to [TYP-009], [TYP-010])

Program	Interpreted	Compiled
<code>List ().ClassName</code>	Undefined property 'ClassName'.	Only instances have properties.
<code>B[0]</code> on an instance	Subscript target should be an ordinal.	Only a collection or a String can be subscripted.
<code>C (1, 2)</code> on a class with no constructor	Expected 0 arguments but got 2.	This class takes no constructor arguments.

Both processors refuse both programs, so nothing runs that should not — but the text differs, and [ERR-002] requires a diagnostic to be the same wherever it is produced.

⚠ The compiled wording is the better of the two in both cases. `Subscript target should be an ordinal.` describes the *subscript* when the fault is the *target*, and it is the message a reader of [TYP-010] meets first. Fixing this should move the interpreter toward the compiler, which is the opposite of the usual direction and is worth saying out loud.

C-8 — An uncaught runtime error carries no Uncaught: prefix. (*loud*)

Interpreted	Uncaught: Index 5 out of range 0..2.
Compiled	Index 5 out of range 0..2.

The message is identical; only the prefix the driver adds is missing. Every conformance case ending in a runtime error meets this, which is why `conformance/0029` carries `// compiled: no`.

⚠ Unlike C-3, this is not a consequence of compiled code lacking line information — the prefix needs nothing the compiled program does not have.

C-9 — A String has a `.Length` property compiled and none interpreted. (*silent*) (refers to [RT-003], [TYP-012])

```
WriteLn ('abc'.Length);
```

Interpreted this is `Uncaught: Only instances have properties.` Compiled it prints `3` .

⚠ **The dangerous direction, as with C-4:** the compiler accepts a program the language refuses. A program developed against the compiler can read `.Length` off a `String` throughout and fail on the first interpreted run, with neither processor having warned that they disagree.

⚠ A collection's `.Length` works in both, so this is specific to `Strings`, and it sits next to the confusion D-15 records — `Length(V)` measures *text* while `V.Length` is a *count*, and here one processor offers a third spelling the other does not have.

Which processor is right is not obvious and is left to chapter 16: a `String` answering `.Length` with a character count is arguably what a reader expects, and it is the only spelling that would agree with a collection's.

C-10 — The compiled back end hoists variables. *(silent) (refers to [DCL-016])*

```
WriteLn (V);  
var V := 7;
```

Interpreted this is `Undefined variable 'V'.` Compiled it prints `nil` .

Every top-level name is emitted at C file scope, so it exists from the start of the program; a variable simply holds `nil` until its initializer runs. The same applies to a class constructed above its declaration.

⚠ **This is the silent direction again, and the worst instance of it.** The compiler does not merely accept a refused program — it substitutes a **value** for a diagnostic, so the program runs to completion with `nil` where a number was meant, and nothing anywhere says so.

⚠ **The compiler is only half wrong.** Hoisting a *function or class* is what [DCL-006] now requires and the interpreter is the defect there (DEF-15). Hoisting a *variable* is what [DCL-016] forbids and the compiler is the defect here. One mechanism, correct for one kind of declaration and not the other, which is why it took a rule split to describe.

C-11 — A top-level block is reordered. *(silent)*

```
WriteLn ('one');  
begin  
    WriteLn ('two');  
end  
WriteLn ('three');
```

Interpreted	Compiled
one two three	one three two

A bare `begin ... end` at the top level runs **in place** interpreted and **after every other top-level statement** compiled. With two such blocks, both are deferred and run in their own order at the end.

⚠ **Statement order is not preserved**, which makes this the most damaging silent divergence recorded. C-6 crashes, which is at least noticeable; C-10 substitutes `nil` for a diagnostic, which a careful reader may spot. This one runs every statement, produces no error, and simply performs them in a different order — so a program whose blocks write files, print, or set variables the rest of the file reads will behave differently under the two processors with nothing to indicate it.

The emitter treats a top-level block as the program's main body, which is right for the one block a program conventionally ends with and wrong for a block appearing anywhere else.

⚠ It also constrains the conformance corpus: any case using a bare top-level block to demonstrate scoping cannot be run under both processors. `conformance/0040` puts its blocks inside procedures for exactly this reason, which keeps the cross-check.

C-12 — A declaration as an unbraced branch or loop body will not compile. (*loud*) (*refers to [STM-002]*)

```
if True then var X := 1;
```

A 'var' as an unbraced branch or loop body is not supported by the C back end yet.

The same program runs interpreted, and the declared name escapes into the enclosing scope.

⚠ **The compiler is right and the interpreter is wrong**, which makes this the only divergence in this annex that [STM-002] resolves by moving the *language* toward the compiler. Fixing DEF-17 removes the entry rather than leaving a gap to be implemented.

C-13 — Two counted for loops sharing a variable name at the top level emit invalid C. (*loud, but in the wrong place*) (*refers to [STM-006], [DCL-008]*)

```
for var I := 0; I < 2; I := I + 1 do Write (I);  
for var I := 0; I < 2; I := I + 1 do Write (I);
```

```
error: redefinition of 'v_I'
```

[STM-006] desugars a counted `for` into a block holding the initializer and a `while`, which is why its variable is scoped [DCL-008]. At the top level the emitter does not open a C block for it, so both loops declare `v_I` in one scope. Inside a function or method it is emitted correctly, which is why the compiler compiles itself — all sixty-eight of its own counted loops are inside one.

⚠ **The emitter breaks its own contract here.** It is supposed to refuse by name what it cannot emit rather than emit something wrong. Instead it produces C that `cc` rejects, so the diagnostic names `v_I` and a line

in a generated file rather than the loop the programmer wrote. A refusal from `algc` would be strictly better than a valid-looking emission that fails downstream.

⚠ It also constrains the corpus: `conformance/0054` puts its loops inside a procedure to keep the cross-check, as `conformance/0040` does for C-11.

C-14 — Compiled code does not check arity. (*silent*) (*refers to [EXP-011]*)

```
function One (A); begin Exit A; end
WriteLn (One (1, 2));
WriteLn ('kept going');
```

Interpreted this is `Uncaught: Expected 1 arguments but got 2`. Compiled it prints `1` and then `kept going` — the extra argument is discarded and the program runs on. Too *few* arguments is accepted as well.

⚠ **This was invisible while** `conformance/0049` **opted out of the compiled half**. It was found within minutes of removing the opt-outs, which is the argument for not having them: an opt-out records nothing and notices nothing, and the case it silences is exactly the case that would have found the bug.

⚠ Silent and unbounded. C-4 accepts a lowercase member name; this accepts any call with any number of arguments, so every arity error in a program compiled by this back end is undetected, and a function reading a parameter that was never passed gets whatever the calling convention left there.

C-15 — A call to an object will not compile. (*loud*) (*refers to [CLS-016]*)

```
object Config;
begin
    function Value (); begin Exit 1; end
end
WriteLn (Config ());
```

A call to `'Config'` is not supported by the C back end yet.

Interpreted the program runs until the call, which raises `Can only call functions and classes`. [CLS-016].

⚠ The program is a **valid** one whose defined behaviour is to raise. The emitter refuses it rather than emitting something that raises, so a program the language merely rejects at run time has no compiled form at all. That is the right way round for a gap — loud, named, and impossible to miss — but it is still a program the two processors do not agree on.

Annex D — advisory notes (*non-normative*)

Where the specified behaviour looks like a mistake. Nothing here weakens the rule it refers to: the body states what the language does, and this annex argues about it. Entries are added as the chapters that expose them are written.

D-1 — Integer overflow is silent. *(refers to [LEX-018], [LEX-033])*

Resolved. The specification now requires an out-of-range literal to be refused when the program is read [LEX-033], and an out-of-range arithmetic result to raise [LEX-018]. The two were separated because they cost differently: a literal is checked once during the scan, while an arithmetic result must be checked on every operation a program performs.

The implementation wraps in both cases, silently and identically in both processors, which is what C does natively. Tracked by DEF-05.

D-2 — ? alone is a valid identifier. *(refers to [LEX-008])*

Resolved. ? and ! are identifier *marks* rather than letters [SRC-005]: they continue an identifier and may not begin one [LEX-008], so Gate? and Send! are single words while ? and ! alone are not identifiers at all.

The implementation classes ? as a letter, so it leads; and it does not admit ! in an identifier at all. Tracked by DEF-03.

D-3 — #0 is constructible but unstorable. *(refers to [LEX-032])*

Resolved by refusing it. #0 is not a Char [LEX-032], and is refused when the program is read exactly as an out-of-range code point is [LEX-025]. That is the smaller of the two available fixes and matches a check the scanner already performs.

The larger fix — giving a String an explicit length so it can hold a zero character — remains the better language, and [SRC-001] already obliges a String to carry a character count distinct from its byte length, so the two changes meet. [LEX-032] is worded so that adopting it later relaxes a restriction rather than reversing a guarantee. Tracked by DEF-08.

D-4 — Widening is refused as firmly as narrowing. *(refers to [VAR-004])*

var X : Integer := 1.5; should certainly be refused: the value does not fit. But var X : Double := 1; is refused on the same terms, and there the value fits exactly and every arithmetic operation in the language already promotes an Integer to a Double when the two meet. A programmer who writes : Double and initializes with 0 is told the types do not match.

Resolved, and generalized. Widening follows Pascal: Integer to Double and Char to String, at any of the six assignment contexts [VAR-017], converting at the point the value arrives. The open question — whether X then holds a Double or an Integer a declaration lied about — is answered explicitly: it holds a Double [VAR-004]. Narrowing stays refused [VAR-018]. Tracked by DEF-10.

⚠ **Why this is a defect and not a later generation.** The question was asked directly, and the answer turns on whether the language *lacks* widening or *has it inconsistently*. It has it: `1 + 1.5` is `2.5` and `'a' + 'bc'` is `abc` today. Only the paths carrying a written type refuse, so a declared type means something narrower than the operators do. An absent facility goes to Annex H — H-1's alternate bases are absent, with nothing in the language to be inconsistent with. One path disagreeing with the rest of the same language is a fault.

⚠ Equality was deliberately left out. Pascal converts in comparisons too, which would make `Copy('abc', 0, 1) = 'a'` true and reverse [LEX-026] — the one part of this that changes what an existing program *means* rather than accepting a program that was refused. It belongs with D-6, in the chapter that specifies `=`.

D-5 — `const` promises less than it appears to. (refers to [VAR-014])

`const C := V;` is legal where `V` is a variable, so a constant's value need not be known before the program runs. The word means only that the binding cannot be reassigned. A reader who takes `const` to mean a compile-time constant — as Pascal's does, and as most languages' do — will be wrong about when the initializer runs and about what the compiler can assume.

Resolved by keeping it. The run-time form is genuinely useful for a value computed once at startup, and nothing here is broken — the word means *this binding may not be reassigned*, which [VAR-014] now states plainly rather than leaving to be inferred. A true compile-time constant, if one is ever wanted, needs a different word rather than a narrowing of this one.

D-6 — Equality and membership disagree. (refers to [VAL-009], [VAL-013])

`1 = 1.0` is true, and `1 in [1.0]` is false. A Map holding the key `1` does not contain `1.0`. Each rule is defensible alone — `=` promotes because arithmetic does, and membership is strict because a hash table cannot be built over a relation that promotes — but together they mean a program can hold two values it calls equal and find only one of them in a collection.

`bootstrap/algol.c` already names this a rough edge in its own comments, which is a fair sign it was noticed and not resolved.

Resolved in favour of `=`. Membership and equality are one relation [VAL-013], paid for by hashing an Integer and a Double of the same numeric value to the same bucket. Making `=` strict instead would have been the far larger change and would surprise every program doing ordinary arithmetic.

⚠ The reason the two diverged is worth keeping: `=` promotes because arithmetic does, and membership was strict because a hash table cannot be built over a promoting relation. The second is a statement about the implementation and the first about the language, which is what decided it. Tracked by DEF-14.

D-7 — `as` is an unchecked assertion. (refers to [VAL-007])

`X as Integer` where `X` holds `'text'` yields `'text'` and raises nothing. The cast silences the checker and is never verified, so the one construct a programmer reaches for when they know more than the checker does is also the one that cannot tell them when they are wrong.

Resolved. `as` is a checked conversion and raises when the value is not of the named type [VAL-007]. The cost falls only on programs that use `as`, which are the programs that asked for the assurance, and a cast that cannot fail is not an assurance at all.

⚠ This decision is what makes [VAR-006] tenable. `as` is the only way a value crosses from untyped into typed, so the strictness there is only reasonable if the crossing is verified; an unchecked cast would have made the boundary a formality and every declared type beyond it a claim nothing had checked. The two rules were decided together and neither stands alone. Tracked by DEF-12.

D-8 — Empty is truthy. *(refers to [VAL-008])*

`0` is falsey, but `0.0`, `''`, `[]` and `[]` are all truthy. So `if not S then` does not mean what a reader coming from most languages will expect it to mean, and `if S.Length = 0 then` is the only reliable spelling.

The rule is at least short to state, and it makes truthiness independent of a value's contents — a collection is a thing, and a thing is there.

Resolved by keeping it. [VAL-008] now states plainly that truthiness is independent of a value's contents, and that `if S.Length = 0 then` is the only spelling that tests emptiness.

⚠ The observation still stands and is **not scheduled**: Integer `0` is the odd one out, and were it truthy the rule would reduce to the genuinely simple "only `nil` and `False` are false". That changes what existing programs mean rather than admitting programs that were refused, so it belongs to neither annex until someone decides to take it — recorded here so the option is not lost.

D-9 — Visibility is advisory. *(refers to [DCL-015])*

`private:` is checked by the type checker, and only where the receiver's type is known. `var C : Any := Counter(); C.Count` yields the private field and raises nothing. A subclass reading its parent's private member by bare name gets it, because a bare name inside a method resolves through `this`, and `this` reduces to no type at all — while the same member through a receiver declared as the parent is correctly refused.

So the guarantee is real for code that annotates its types and absent for code that does not, which is the opposite of where a guarantee is most wanted. It is also not a security property and was never meant to be: the checker is advisory by design in a gradually typed language.

⚠ Any fix collides with a deliberate decision recorded in the checker: `this` reduces to nothing precisely so that a class's own code is free of the visibility rule. Typing `this` as its class would make the bare-name case work and would also make every `this.Private` inside a subclass an error the language currently allows.

Resolved by saying so plainly. [DCL-015] now states normatively that `private:` is advisory — an intention that buys a diagnostic wherever types are written, and not a boundary a program may rely on.

⚠ The alternative was enforcing it at run time on the instance, where the class is always known. That closes the hole completely and was the earlier recommendation here, but it puts a check on **every property access**

— which is the cost the whole type-system direction of this specification exists to avoid. [VAR-006] tightened declarations so the C back end could trust a declared type and emit without runtime checks; paying one back here, on the most frequent operation a program performs, would trade away more than it buys.

Nothing is lost that was ever really held: the guarantee was already absent whenever a type was omitted, and the change is to stop implying otherwise.

D-10 — Integer division by zero raises; Double division by zero does not. *(refers to [EXP-006])*

`1 / 0` is the runtime error `Division by zero`. `1.0 / 0` is `Infinity`, and `0.0 / 0` is `NaN`, and neither stops the program. So whether dividing by zero is a bug or a value depends on which numeric type reached the operator — and [EXP-005] means an Integer becomes a Double whenever it meets one, so the same expression can change category with an edit far from it.

The Double behaviour is IEEE 754 and is what C does for free; the Integer behaviour has no such answer available, since there is no integer infinity to produce.

Resolved by keeping both. [EXP-006] now states the asymmetry as a rule rather than leaving it to be discovered, and says why: each behaviour is correct for its own type. Raising on Double division would depart from IEEE 754 for no gain, and returning a value for Integer division would have to invent one, since there is no integer infinity.

⚠ The genuinely surprising part is not the asymmetry but its reach: [EXP-005] promotes an Integer whenever it meets a Double, so an edit far from a division can move it from the raising category to the value-producing one. That is recorded in the rule.

D-11 — A parameter's declared type is decoration on a function and a contract on a method. *(refers to [FUN-006], [FUN-007])*

`function F(N : Integer)` accepts a String, a Double or a Boolean without complaint. The same signature as a method refuses all three with `No matching signature for function`. Nothing in the language says the two differ, and the annotation looks identical in both places.

The cause is structural rather than deliberate: a method goes through overload selection, which compares whole signatures, and a top-level function does not overload [see 8.3] so nothing ever compares its parameters. Only the arity is checked.

⚠ Note the direction of the surprise. A declared return type IS enforced [FUN-008], so within one declaration the result is checked and the arguments are not — which is the reverse of what a reader would guess, since arguments come from outside and are the less trustworthy of the two.

Resolved. [FUN-006] requires parameter types to be enforced on every call, top-level subprogram or method alike. The check already exists in `Fits`; what is missing is calling it when there is nothing to select between. Tracked by DEF-19.

⚠ This was not really an open question by the time it was reached: [VAR-017] had already listed a parameter as one of the six assignment contexts, so [FUN-006] as written contradicted a decided rule. The direction-of-

surprise note above still stands and is worth keeping — a declared *return* type is checked while the arguments are not, which is the reverse of what a reader would guess.

D-12 — Inheriting from a non-class reports the wrong thing. *(refers to [CLS-014])*

```
var X := 1;  
class C (X);  
begin  
end
```

fails with `Only instances have properties.` — a sentence naming neither the class, nor the superclass, nor inheritance, and describing a property access the program never wrote. The message belongs to the machinery rather than to the mistake.

 **Corrected.** This entry previously said the check happens where the superclass is *used* rather than where it is declared. It does not: the declaration above is refused even when the class is never constructed. Only the wording is at fault, which makes this smaller than it was recorded as being — and is why it was worth running rather than reasoning about.

Diagnostics are part of the observable surface [1.2], so this is a specified behaviour and not merely a rough edge — a conforming implementation must reproduce the misleading sentence exactly.

Resolved. [CLS-014] requires the shape the other inheritance errors already use — `'X'` is not a class. , beside A class can't inherit from itself. The check already happens in the right place, so this is a message and nothing else. Tracked by DEF-20.

D-13 — Truthiness reads a value a program cannot. *(refers to [ENU-009], [ENU-010])*

The first member of every enumeration is falsey, because truthiness reads the member's ordinal — and the ordinal is not reachable from the language. `RED` is false and `GREEN` is true, and a program can discover which is which only by testing each for truth.

So `if Colour then` depends on where a member sits in a list that the program cannot inspect, and reordering an enumeration's members silently changes the truth of every conditional written over it. Nothing warns, because nothing about the declaration looks conditional.

The rule exists so that enumerations behave like the small integers they are represented by, which is a real convention and not an accident.

Resolved, in both halves and in opposite directions.

[ENU-010] requires a member to answer `Ordinal` , its zero-based position, so the value that governs the behaviour can be read and compared. `Ordinal` already exists on the implementation's own class and simply is not published. Tracked by DEF-21.

[ENU-009] is **kept**, and stated as a feature rather than tolerated as a quirk — position-based truthiness is what makes `(Off, On)` and `(No, Yes)` usable directly in a condition. The entry below records the reasoning,

which began as an argument for changing it and ended as the argument for keeping it.

⚠ **[ENU-009] is settled: it stays.** The case against it was: an enumeration member is not a number, nothing else in the language makes a declared name falsey by position, and reordering an enumeration silently changes the truth of every conditional written over it.

⚠ **The case for it is stronger, and was missed when this entry was written.** Position-based truthiness is what lets a program declare its own two-valued types and use them directly in a condition:

```
type Flag    = (Off, On);
type Answer = (No, Yes);
```

`if F then` reads correctly for both, with no comparison and no conversion. The convention that falls out — put the absent, off or zero member first — is already the one this compiler follows in `FUN_NONE` and `CLASS_NONE`. Under that reading the rule is a feature with a discipline attached, not a trap, and the discipline is the same one a `case` statement already asks for.

⚠ **And the original complaint is half answered.** This entry's title says truthiness reads a value a program *cannot* — but [ENU-010] now requires the ordinal to be readable. Position-based truthiness over a *visible* position is a stated rule rather than hidden machinery, which is most of what was wrong with it.

⚠ **A further prospect, TABLED and separable: it could remove the built-in Boolean,** leaving `type Boolean = (False, True)` as an ordinary enumeration. This is not decided and is not scheduled. Two costs argue for keeping the two questions apart:

- **[ENU-011] collides with it.** Members bind bare, so any program declaring an enumeration with a `True`, `False`, `Yes` or `No` member would make that bare name ambiguous — and those names appear in almost every program. Boolean would have to be exempt from the ambiguity rule, which is a special case reintroduced one level down.
- **Representation.** `VAL_B00L` is a distinct runtime tag while an enumeration member is an interned object, so this puts an indirection on the most common value a program has. That runs against the reason [VAR-006] was tightened.

`true` and `false` are also keywords [LEX-010], which would drop from 37 to 35.

Changing it would reverse part of [VAL-008], which the conformance pass decided in chapter 7, so it is recorded here rather than taken unilaterally. Evidence gathered for whoever decides: the compiler's own two enumerations are compared explicitly at all five of their use sites — `if CurrentFunction = FUN_NONE then` — and never tested for truth bare, so the change is safe against the largest body of Algol-24 that exists.

D-14 — Circular imports fail, and say something else. (*refers to [MOD-012]*)

⚠ **This entry was largely wrong, and running it is what showed that.** It said a cycle between two modules fails with `Type mismatch!` It does not: cycles between modules **work**, and so do cycles of three or more [MOD-012].

The `Type mismatch!` came from the probe's own fixtures, which returned `'A'` and `'B'` from functions declared `: String`. A one-character literal is a `Char` [LEX-023], so those functions failed whether or not any cycle existed — the recording was evidence of DEF-10 and was read as evidence about modules. The fixtures now return two-character strings and the probe records `Ay`.

What remains true is the **root** case [MOD-014]: a module importing the file being run leaves the root's own imported names undefined, after the root's body has already printed. Compiled, the same shape refuses with `Two modules named 'X' (C-1)`, which at least names the right subject.

So there is one shape of problem rather than three, and it is the one this repository already knew about — the root is never entered in the loader's map, so it is parsed twice.

Resolved. [MOD-012] states that cycles between modules work, which they do. [MOD-014] requires the root case to work the same way, and DEF-24 tracks the distance.

The fix is to give the root a module identity, so `a` uses pointing back at it resolves to the copy already loaded — what [MOD-003] does for every other file and what makes [MOD-012] work. That is more than a guard: an import-only node carries no statements, and the importer genuinely needs the root's exports, so the root must be registered with its environment **before its own body runs**.

Refusing the root cycle by name would be an improvement on the present diagnostic and is much the smaller change, but it settles for less than [MOD-014] asks.

⚠ **Not hypothetical for this repository, and narrower than it once read.** `compiler/Parser.a24` uses `Interpreter`, which uses `Parser` — a cycle between two modules, which works, interpreted and compiled alike. The compiler compiles itself and reaches a fixed point.

What fails is making a file in that cycle the **root**: `algc --compile compiler/Parser.a24` refuses with `Two modules named 'Parser' is not supported by the C back end yet.` [Annex C, C-1].

`compiler/Main.a24` is unaffected because nothing imports it, which is why the whole suite compiles. An earlier version of this entry said the compiler's own source could not be compiled by itself; that was wrong.

D-15 — Two different things are spelled `Length`. *(refers to [RT-003])*

```
var L := [10, 20, 30];
L.Length      → 3
Length(L)     → 12
```

The property answers the collection's count. The function stringifies its argument and measures the text, so `Length(L)` is the length of `[10, 20, 30]`. `LengthNative` is literally `Exit Length(Str(Arguments[0]))`.

⚠ The failure mode is the bad one: the wrong call returns a **plausible number** rather than an error. A program asking `Length(L)` of a collection gets an answer, uses it, and is wrong.

⚠ **Corrected.** This entry claimed the two coincide for small lists of one-digit numbers before diverging. They never coincide: a List of n one-digit numbers renders as `[1, 2, 3]`, which is $3n$ characters, against a count of n — so $0/2$, $1/3$, $2/6$, $3/9$. The claim was plausible and wrong, and the rule is no weaker without it.

The function is right for its intended argument. `Length('abc')` is 3, which is what a Pascal programmer expects, and the collection property is right too. Only the shared name is wrong.

Resolved. [RT-003] refuses `Length` of a collection — `Length` expects text; use `.Length` for a collection. A program that means the count says so, and one that means the rendering writes `Length(Str(L))`, which is what it is getting today by accident. Tracked by DEF-25.

⚠ Verified safe against the largest body of Algol-24 that exists: every `Length(...)` call in `compiler/*.a24` is on text.

⚠ [RT-017] closes the other half of the confusion by giving a `String` the same `.Length` property every collection has, so `.Length` becomes the uniform spelling for "how many" and `Length(...)` is left meaning only "how long is this text".

D-16 — `Val` always yields a Double, and `Max` never accepts one. (refers to [RT-009], [RT-010])

`Val('42')` is `42.0`, not `42`, so text that plainly holds an integer cannot be parsed into one — and the result then cannot be passed to `Max`, which refuses anything but `Integers` with `Max` expects `Integers`. The two built-ins are individually defensible and jointly unusable: `Max(Val(A), Val(B))` fails for every input.

Given [VAR-004], which refuses `var X : Integer := 1.5;` and even `var X : Double := 1;`, a program has no smooth path from parsed text to an `Integer` at all.

Resolved, both halves. [RT-009] makes `Val` answer an `Integer` where the text has no point and a `Double` where it has one, reading the same characters the literal rules do [LEX-015], [LEX-020]. [RT-010] lets `Max` take any two numbers, promoting as every other numeric operator does [EXP-005].

⚠ They are **one** defect, DEF-27, rather than two. Either change alone helps, but only both together make `Max(Val(A), Val(B))` — which fails for every input today — work at all.

D-17 — A type error says only "Type mismatch!" (refers to [ERR-006])

Every mismatch the checker finds produces the same five words, with no file, no line, no token and neither of the types involved. In a seven-line program the diagnostic is:

```
Uncaught: Type mismatch!
```

and nothing else. The parser and resolver, by contrast, print the file, the line, the source text and a caret under the offending token [ERR-005] — so the machinery for a good diagnostic already exists, is already used two phases earlier, and the checker simply does not reach for it.

⚠ This compounds with [ERR-002]: because a type error stops the program before any statement runs, a programmer gets no output to orient by either. The message is the only information available, and it carries none.

⚠ And with [FUN-006]: a parameter's declared type is unenforced, so mismatches surface in fewer places than a reader expects — which makes the ones that do surface harder to locate, not easier.

Resolved. [ERR-006] now requires the three-line excerpt a parse error carries [ERR-005] and both type names — `Expected Integer, found String`. The token is in hand at every one of the five sites that raise this, so the information is discarded rather than absent, and `Console.Error` already produces the shape two phases earlier. Tracked by DEF-29.

⚠ This remains the single cheapest improvement to the language's usability in this annex, and it grew cheaper while the conformance pass ran: DEF-19 will make parameter types enforced, so mismatches will surface in *more* places than they do now — each of them reported by five words that say nothing.

D-18 — `AssertTrue` reports a comparison it did not make. (*refers to [TST-012]*)

`AssertTrue(False)` fails with:

```
Assertion 'left = right' failed.
```

There is no left and no right. The message belongs to `AssertEqual`, which appends `Expected 'E' but got 'A'`. to the same stem, and `AssertTrue` reuses the stem and supplies no operands — so the reader is told an equality failed when a truth test did, and given nothing about what was actually false.

`Fail(M)` produces `Failed. M`, with two spaces, which is a third shape again.

Resolved, and further than recommended. The phantom stem goes from *every* form, not only from `AssertTrue`: `'left = right'` names operands that no message ever fills in, and reads like a template left unfinished. [TST-012] now specifies `Assertion failed.` followed by two spaces and a clause that carries the actual values — `Expected true but got 'V'`. for `AssertTrue`, and `Expected 'E' but got 'A'`. for `AssertEqual`. Tracked by DEF-30.

⚠ The type-naming form was **missing from the table entirely** and is kept: a `Char` and a `String` both render as `3`, so a message quoting only the rendered values would read `Expected '3' but got '3'`. Both processors already produce it. Documenting it was as valuable as fixing the stem.

⚠ Any change here alters the report, which [TST-008] specifies and which both processors must reproduce byte for byte — so it is a change to the observable surface and to `bootstrap/algol.c` in the same breath, not a cosmetic edit.

Annex E — what could be written in Algol-24 itself (*non-normative*)

The collections and the built-in functions are native today. This annex asks, for each, whether it is native because it *must* be or only because it always has been — and what one feature would have to be added to unbind it.

⚠ This is the plan, not an idle question. The intent is to move the collections out of the core language and into a unit written in Algol-24 wherever they are not genuinely pinned, and this annex is the survey of what that costs. Annex H, H-9 carries the work; chapter 14 records the behaviour the unit will have to reproduce.

The reason is the one below, and it is worth more than the tidiness: everything moved out of the runtime is one less thing the C back end and the interpreter can disagree about — and Annex C is now fifteen entries long.

⚠ Three rules do most of the pinning, and they are worth naming once rather than repeating: a class cannot be subscripted [TYP-010], cannot be iterated [TYP-011], and cannot expose a computed property [TYP-012]. A collection written in Algol-24 would work, but it would be written `L.Get(0)` , `L.Length()` and a loop over an index — a second-class citizen beside the built-ins.

E.1 The collections

Array — pinned, and rightly so.

`Array` is fixed-size, indexed in constant time, and holds arbitrary values. Nothing in the language can express that. `Buffer` stores bytes rather than values; a linked structure of class instances gives $O(n)$ access; and a class cannot be subscripted in any case. It is the one collection that is genuinely primitive.

Recommendation: keep it native, and treat it as the primitive the others are built on rather than as one collection among five.

Stack — unbound already, in all but syntax.

A `Stack` is `Push` , `Pop` , `Peek` and a count over a sequence. It can be written in Algol-24 today, in full, as a class holding a `List` — and has been: `spec/probes/TYP-012-stack-and-set-in-algol24.a24` is a working one, beside a `Set` whose membership test is a hand-written scan using no native `Contains` , and a `Mod` built from arithmetic alone. All three behave as the built-ins do.

Nothing about it is primitive: no literal claims its name, no subscript is needed, and its whole surface is method calls.

The only visible difference would be `S.Length()` against the built-in `S.Length` — [TYP-012] alone.

Recommendation: the best first candidate to move into a unit. It would prove the path with the least at stake, and the only thing it waits on is a getter.

Set — writable today; would want hashing later.

`Set` needs membership. Over a `List` with a linear scan it can be written immediately, and the probe named above does exactly that. A hashed version is also expressible: `Ord` yields a character's code point as

an Integer — `Ord('A')` is 65 — so a string hash can be computed in the language without reaching for anything native.

Nothing pins it syntactically. `Set(L)` is an ordinary constructor call, not a literal form.

Recommendation: movable, and the performance question is separable — ship the linear version as a unit, hash it later if it matters.

List — pinned by its literal, not by its behaviour.

A growable sequence over `Array`, doubling when full, is straightforward Algol-24. What pins `List` is syntax: `[1, 2]` produces one, so the language itself hands the name out. Subscripting `[TYP-010]` and `for ... in [TYP-011]` would also have to be lifted, or every use would read `L.Get(I)`.

Recommendation: unbinding this one is worth doing only if user-defined subscript and iteration arrive first. Otherwise the unit is strictly worse to use than the built-in it replaces.

Map — pinned by its literal, doubly.

`[:]` and `[k:v]` produce a Map, so the name is claimed the way `List`'s is, and the natural `M[K]` needs `[TYP-010]` as well. The implementation itself is ordinary: two sequences, or `Array` buckets with an `Ord`-based hash.

⚠ Insertion order is specified behaviour, not an accident of the implementation, so any replacement must keep it.

Recommendation: same as `List`, and after it. The literal is the harder half: a language that lets a unit claim `[:]` is a much larger language.

E.2 The built-in functions

Native	Could it be written in Algol-24?
<code>Max</code> , <code>Mod</code>	Yes, today, from arithmetic alone.
<code>Copy</code> , <code>Pos</code>	Yes, from <code>Length</code> and subscripting a <code>String</code> , at a cost in speed.
<code>Val</code>	Yes, given <code>Ord</code> for digits.
<code>Ord</code> , <code>Char</code>	No. They convert between a character and its code point, and nothing else reaches that representation.
<code>Length</code>	No, for <code>String</code> . A user type's own length is <code>[TYP-012]</code> .
<code>Str</code>	No. Rendering a <code>Double</code> in the specified shortest round-trip form needs the value's bits, which the language cannot see.

Native	Could it be written in Algol-24?
clock , TextFile , FileExists , ParamCount , ParamStr	No. The operating system is not otherwise reachable.
Buffer	No. It is a memory primitive with an explicit lifetime.
Write , WriteLn	No. Output is not otherwise reachable.

E.3 The one feature that unbinds the most

Of the three pins, [TYP-012], **the missing getter, is the cheapest and buys the most**. It is a declaration form rather than a semantic change; it makes `Stack` and `Set` writable as units indistinguishable from the built-ins; and it is needed by ordinary user classes regardless of whether any collection ever moves.

Subscript [TYP-010] is next, and iteration [TYP-011] after it — those two together are what `List` and `Map` wait on, and both are larger changes, because each means dispatching a language construct into user code.

Annex F — defects (*non-normative*)

Where the implementation does not do what this specification requires. Each entry names the rules it violates, what the implementation does today, and the program in `defects/` that reproduces it.

⚠ A defect's test is a **reverse conformance test**: it records the wrong behaviour and passes while that behaviour persists. It turns **red when the defect stops reproducing**, because a fix is as much a change to be noticed as a regression — and a suite that is permanently red is a suite nobody reads.

DEF-01 — Text is bytes, not characters. (*violates [SRC-001], [SRC-002], [SRC-004], [SRC-005]*)

The implementation treats source and strings as bytes. `Length('café')` is 5 rather than 4; `'café'[3]` is the first byte of a two-byte sequence rather than `é`; and the scanner refuses every non-ASCII byte outside a comment or literal, so a Unicode letter cannot appear in an identifier.

Reproduce: `defects/DEF-01-text-is-bytes.a24`

Scope of the fix. This is the largest change the specification asks for, and it is not confined to one place:

Where	What changes
Scanner	admit Unicode letters in identifiers; decode UTF-8
Runtime, both	a String gains a character count distinct from its byte length
<code>Length</code> <code>Copy</code> <code>Pos</code> <code>subscript</code>	count and index characters

Where	What changes
Ord Char	full code-point range, not 0 ... 127 [LEX-025]
Emitter	identifiers mangled per Annex G

⚠ It also decides D-3 on the way past: a String that carries its own length can hold `#0`, and the truncation recorded there stops being possible.

DEF-02 — Identifiers are matched case-sensitively. (*violates [SRC-011]*)

`Xyz` and `xyz` are two names where the specification says they are one, and declaring both in one scope is accepted where it should be a duplicate [VAR-007].

Reproduce: `defects/DEF-02-identifiers-are-case-sensitive.a24`

Scope of the fix. Every name lookup folds ASCII case — variables, fields, methods, unit-qualified names — while the lexeme is preserved for diagnostics. `Scanner.a24` already folds for keyword lookup and `algol.c` already has `alg_stricmp`, so the machinery exists in both.

⚠ **Prerequisite.** `compiler/Console.a24` declares a module-level `const INFO` and an object member procedure `Info`, and likewise `ERROR` and `Error`. Under folded lookup the member would shadow the constant inside `Info` itself and the compiler would break. Those two constants must be renamed before this defect is fixed.

⚠ This also **reverses C-4**. Collection member names are matched case-insensitively by the compiler and exactly by the interpreter; under [SRC-011] the compiler is right and the interpreter is the one to change. That is the safer direction — it can only make programs start working.

DEF-03 — Identifier marks are wrong in both directions. (*violates [SRC-005], [LEX-007], [LEX-008]*)

`?` is classed as a letter rather than a mark, so it may begin an identifier and `var ? := 7;` declares a variable. `!` is not admitted in an identifier at all, so `Send!` is refused as an unexpected character.

Reproduce: `defects/DEF-03-identifier-marks.a24`, `defects/DEF-03b-bang-is-not-an-identifier-mark.a24`

⚠ **Two files, because the two halves cannot share one.** A program that uses `!` is refused during the scan, which ends the run before anything else in it can be observed — so the `?` half would never execute. This is the general shape of the problem: a defect in the scanner admits only one observation per program.

Scope of the fix. `IsAlpha` stops answering true for `?`; a new predicate answers for `?` and `!`; `ScanIdentifier` accepts a mark only after the first character. Annex G's mangling already maps both marks.

DEF-04 — `print` is a keyword and a statement. (*violates [LEX-010], [STM-022]*)

The scanner registers `print` as a thirty-eighth keyword and the parser has a `print E` statement. Neither is part of the language, and the word cannot be used as a name: `var print := 7;` is refused with `Expect variable name.`

Reproduce: `defects/DEF-04-print-is-a-keyword.a24`

Scope of the fix. Remove the entry from `Keywords`, `TOKEN_PRINT` from `TokenType.a24`, the statement from the parser, `VisitPrintStmt` from the interpreter and the resolver, and its case from the emitter. ⚠️ The compiler's own sources must not use it first.

⚠️ **One of them does.** `compiler/Main.a24` 's `SAMPLE` — the program `algc` runs when given no arguments — is written with `print`, so it stops working the moment the statement is removed. It must be rewritten to use `WriteLn` as part of this fix, not after it.

DEF-05 — Integer overflow is silent. (*violates [LEX-018], [LEX-033]*)

An out-of-range literal wraps instead of being refused, so `2147483648` is `-2147483648` and `999999999999999` is `276447231`. Arithmetic wraps instead of raising, so `2147483647 + 1` is `-2147483648`. Both processors agree, because both let C's native wrapping through.

Reproduce: `defects/DEF-05-integer-overflow-is-silent.a24`

Scope of the fix. The literal half is small and local: `ToInteger` in the scanner learns the range and records a scan error. The arithmetic half touches every operator in `VisitBinary` and in `alg_add` and its neighbours, and it is the half that costs at run time.

DEF-06 — A Char is limited to 0 ... 127, and says so in the wrong shape. (*violates [LEX-024], [LEX-025]*)

`#233` is refused where the specification requires `é`. The refusal also has the wrong form: `ScanChar` builds the value by calling the `Char` built-in during the scan, so a range failure surfaces as a bare `Char` is limited to `0..127`. with no line number and no source caret, unlike every other scan error [ERR-004].

Reproduce: `defects/DEF-06-char-range-and-diagnostic.a24`

⚠️ The diagnostic half is worth fixing **whatever** the range turns out to be. It is not really about Unicode; it is a scan error escaping as a raise.

Scope of the fix. The range moves with DEF-01, since a `Char` wider than a byte and a `String` of characters are the same change. The diagnostic is independent: `ScanChar` checks the range itself and records an error rather than calling a built-in that raises.

DEF-07 — An unterminated string reports the wrong line. (*violates [LEX-031]*)

The diagnostic names the line the scan *reached* — the end of the file — rather than the line the string opened on. The two differ by however much text follows the stray quote, and the line reported is the one place the fault certainly is not.

Reproduce: `defects/DEF-07-unterminated-string-line.a24`

Scope of the fix. `ScanString` records the line at the opening quote and uses it in the message. `Line` is already tracked; nothing new has to be measured.

DEF-08 — `#0` is accepted and truncates a String. (*violates [LEX-032]*)

`#0` is `Char` is true where the specification refuses the literal outright, and a `String` silently truncates where one is concatenated in, so `Length('a' + Str(#0) + 'b')` is 2. Both processors agree, because both represent a `String` as C does.

Reproduce: `defects/DEF-08-nul-char-truncates.a24`

Scope of the fix. The same range check as DEF-06, extended downwards. See D-3 for why refusal rather than storage is what the rule asks for, and why the larger fix stays available.

DEF-09 — A written type is enforced on a declaration and not on an assignment. (*violates [VAR-006]*)

`var I : Integer := A;` is refused where `A` is `Any` or untypeable, and `var I : Integer;` followed by `I := A;` is accepted — the same value reaching the same variable one line later. The strict check lives on the declaration only; the assignment path calls `Assignable`, which permits `Any` in both directions.

Reproduce: `defects/DEF-09-assignment-escapes-the-type.a24`

⚠ **The permissive path is the wrong one to keep.** It is tempting to read this as the declaration being too strict, because the declaration is what produces a diagnostic. But the assignment is where an untyped value enters a typed variable with nothing checking it, which is precisely what [VAR-006] exists to prevent.

Scope of the fix. The `AssignExpr` and `SetExpr` cases in `compiler/TypeChecker.a24` apply the same rule the declaration does — an actual type of `''` or `Any` does not satisfy a written one. ⚠ The compiler's own sources must be made to pass first, and they are the largest body of Algol-24 that exists; expect casts to be needed at boundaries that currently have none.

DEF-10 — Widening is refused wherever a type is written. (*violates [VAR-004], [VAR-017], [EXP-014]*)

Neither widening pair is applied at an assignment context. `var X : Double := 1;` , `var S : String := 'a';` and an `Exit 1` from a function declared `: Double` are each `Type mismatch!` Narrowing is correctly refused.

⚠ **A parameter is an assignment context too**, so the same gap shows up in overload selection: with only a `String` overload declared, `Take('a')` is `No matching signature for function.` where the `Char` should widen. Exact matching works and is correctly preferred, so only the fallback is missing.

Reproduce: `defects/DEF-10-widening-is-refused.a24` , `defects/DEF-10b-widening-at-a-parameter.a24` , and `conformance/0025-operators-widen.a24` for the other half.

⚠ **The two files are the defect.** One shows `1 + 1.5` and `'a' + 'bc'` widening; the other shows the same conversions refused a line after a type is written. Neither is remarkable alone. The check is whole-

program and compile-time, so they cannot share a run — the same one-observation-per-program constraint as DEF-03.

Scope of the fix. Assignable admits Integer where Double is expected and Char where String is, and both processors convert **at the point of assignment** so the variable holds the wider type rather than one the declaration misdescribes. The conversions themselves already exist in both runtimes, since the operators perform them; this is a matter of calling them from one more place.

DEF-11 — of parses only after List . (violates [VAR-008])

Map of Integer , Set of Integer and Array of Integer are each refused with Expect ';' after variable declaration. , so only a List may carry an element type.

Reproduce: defects/DEF-11-of-is-list-only.a24

Scope of the fix. The of clause is parsed for any collection type rather than one. ⚠ The checker must then decide what a Map of T means — the value type, almost certainly, since that is what a subscript and a Get yield — and say so in [VAR-016] before the parser admits the syntax.

DEF-12 — as is unchecked. (violates [VAL-007])

X as Integer where X holds 'text' yields 'text' and raises nothing. The cast silences the checker and verifies nothing.

Reproduce: defects/DEF-12-as-is-unchecked.a24

⚠ **This is DEF-09's prerequisite, not a separate errand.** [VAR-006] sends every untyped-to-typed crossing through as ; if as does not check, tightening the assignment path only moves the hole rather than closing it. Fix this one first.

Scope of the fix. The cast becomes a runtime operation in both processors rather than a checker-only annotation: it tests the value against the named type and raises on failure, with nil passing every cast [VAR-005].

DEF-13 — An unknown type name in is is silently false. (violates [TYP-013])

1 is Nonexistent is false rather than being refused. The name is never resolved, so a misspelled type answers false and the branch it guards never runs — if X is Integr then takes the else arm for every X , and the program reports success.

Reproduce: defects/DEF-13-unknown-type-name-is-silent.a24

⚠ The same undefined name used as a **value** is Undefined variable , so this is an inconsistency in one operator rather than a general looseness about names.

Scope of the fix. The resolver already walks every IsExpr for the receiver; it checks the type name against the declared types and records an error when it denotes nothing. ⚠ Any must stay legal there even though X is Any is always false [VAL-005] — it names something, it just never matches.

DEF-14 — Membership does not follow equality. (violates [VAL-013])

`in`, `Contains` and `Map` key lookup compare strictly where `=` promotes, so `1 in [1.0]` and `1.0 in [1]` are both false and a `Map` holding the key `1` does not contain `1.0` — while `1 = 1.0` is true. A program can hold two values it calls equal and find only one of them in a collection.

Reproduce: `defects/DEF-14-membership-does-not-follow-equality.a24`

Scope of the fix. `strict_equals` in the C runtime and the corresponding path in `ObjCollection.a24` use the promoting comparison. ⚠ **The hash is the real work**, not the comparison: a `Map` and a `Set` bucket by a hash, and an `Integer` and a `Double` of the same numeric value must land in the same bucket or the lookup will not find what the comparison would have matched. Changing the comparison alone would leave `Contains` answering false for a key the `Map` holds.

DEF-15 — Functions and classes are not hoisted. (violates [DCL-006])

A call above the declaration is `Undefined variable 'F'.`, and constructing a class written below is `Undefined variable 'Dog'.`, so a file must be organized bottom-up. The compiled back end already does what the rule requires.

Reproduce: `defects/DEF-15-declarations-are-not-hoisted.a24`

⚠ **Only the interpreter is wrong, and only about half of what it does.** The same mechanism in the compiler also hoists *variables*, which [DCL-016] forbids — recorded separately as C-10. Fixing this defect must not be done by adopting the compiler's behaviour wholesale.

Scope of the fix. Top-level function and class declarations are bound in a pass before the statements run, in the same way `HoistTests` already walks a file ahead of execution. ⚠ **Only functions and classes:** a `var` keeps its initializer at the point it is written, so binding it early would substitute `nil` for a diagnostic.

DEF-16 — `as` binds too loosely. (violates [EXP-003])

`as` sits below `and` and `or` rather than with the unary operators, so `A and B as C` casts the whole conjunction, and `A as Integer > 3` does not parse at all — the cast consumes the expression and the comparison is left over, giving `Expect '))'` after arguments.

Reproduce: `defects/DEF-16-as-binds-too-loosely.a24`

⚠ **Only the parse failure is reproducible today.** The other symptom — a cast covering a conjunction — prints the same value under either precedence, because a cast has no runtime effect (DEF-12) and `and` yields an operand. It becomes observable the moment DEF-12 is fixed.

⚠ **This mattered less before `as` was checked.** A cast covering a conjunction had no runtime effect and was merely an oddity in the checker; under [VAL-007] it raises when the conjunction is not of the named type. Fixing DEF-12 without this one makes a latent oddity into a live fault.

Scope of the fix. `as` moves from its own level in `Parser.a24` 's expression chain to the unary level. ⚠ Not purely additive: `A and B as C` changes meaning. Nothing in `compiler/*.a24` uses `as` next to a binary operator, so the change is safe there, but it is a change to what an existing program means rather than only to what parses.

DEF-17 — A declaration is accepted as an unbraced branch or loop body. (*violates [STM-002]*)

`if C then var X := 1;` is accepted, and the declared name escapes into the enclosing scope — so whether `X` exists is decided by the branch, and reading it after a false condition is `Undefined variable 'X'`. A loop body never entered behaves the same way.

Reproduce: `defects/DEF-17-declaration-as-an-unbraced-body.a24`

⚠ **Fixing this removes C-12 rather than implementing it.** The C back end already refuses the construct, so the language, the compiler and the interpreter end up agreeing — the only entry in Annex C of which that is true. Every other divergence there needs the compiler brought up to the language.

Scope of the fix. Statement in `compiler/Parser.a24` refuses a declaration where a branch or loop body is expected, with a message of its own rather than the emitter's "not supported yet" wording — the construct is not unimplemented, it is not part of the language.

DEF-18 — A procedure may `Exit` a value. (*violates [FUN-003]*)

`procedure P (); begin Exit 7; end` is accepted, and `WriteLn (P ())` prints 7, so `function` and `procedure` are interchangeable and the word documents nothing.

Reproduce: `defects/DEF-18-a-procedure-may-exit-a-value.a24`

Scope of the fix. The parser knows which word opened the declaration; `Exit` with a value inside a procedure is refused there. ⚠ Verified safe against the largest body of Algol-24 that exists: **no procedure in** `compiler/*.a24` **exits a value**, so the compiler's own sources need no change.

DEF-19 — A top-level subprogram's parameter types are not enforced. (*violates [FUN-006], [VAR-017]*)

`function G (N : Integer)` accepts a `String`, a `Double` or a `Boolean`. Only the arity is checked, because a top-level subprogram does not overload and nothing ever compares its parameters — while the same signature as a **method** refuses all three.

Reproduce: `defects/DEF-19-top-level-parameter-types-unenforced.a24` , with `refusals/0025-method-parameter-type-is-enforced.a24` for the contrast.

⚠ Not an open question by the time it was reached: [VAR-017] already listed a parameter as one of the six assignment contexts, so this was a contradiction to propagate rather than a decision to make.

Scope of the fix. The comparison exists in `Fits` ; what is missing is calling it from the plain-call path where there is nothing to select between. ⚠ It should land with DEF-10, since a parameter must **widen** as well as match — a `Char` argument reaching a `String` parameter is correct and must not start failing.

DEF-20 — Inheriting from a non-class reports the wrong thing. (violates [CLS-014])

```
var X := 1;  
class C (X);  
begin  
end
```

is refused with `Only instances have properties.` where [CLS-014] requires `'X'` is not a class. The sentence names neither the class, nor the superclass, nor inheritance, and describes a property access the program never wrote.

Reproduce: `defects/DEF-20-superclass-message-is-wrong.a24`

⚠ Smaller than it was first recorded as being. Both [CLS-014] and D-12 said the check happens at construction; it does not. The declaration above is refused even when the class is never constructed, which the reproduction demonstrates with a `WriteLn` on either side of it. Only the wording is at fault — and that correction came from running the case, not from reading the code.

Scope of the fix. `VisitClassStmt` in `compiler/Interpreter.a24` raises its own message rather than falling through to the property-access path.

DEF-21 — An enumeration member's ordinal is not reachable. (violates [ENU-010])

`RED.Ordinal` is `Only instances have properties.` A member answers no property at all, so its position cannot be read — while that same position decides the member's truthiness [ENU-009]. A program can discover whether a member is falsey only by testing it for truth.

Reproduce: `defects/DEF-21-enum-ordinal-is-not-reachable.a24`

⚠ The ordinal is also what a program needs in order to put members in an order [ENU-008], to index an array by one, or to write one out and read it back. None of that is reachable today.

Scope of the fix. `ObjEnum` already carries the position; `Get` answers `Ordinal` from it, in the way `ObjInstance` answers `ClassName` [CLS-008], and the C runtime does the same in `alg_property`.

⚠ This is the smaller half of D-13. The larger half — whether the first member of an enumeration should be falsey at all — is still open, because changing it reverses part of [VAL-008], decided in chapter 7.

DEF-22 — Two enumerations may not share a member name. (violates [ENU-003], [ENU-011])

`type First = (A, B);` followed by `type Second = (A, C);` is refused with `'A' is already defined.`, so adding a member to one enumeration can break an unrelated one elsewhere in the program — and the qualified form `First.A`, which would be unambiguous, never gets a chance to help.

Reproduce: `defects/DEF-22-shared-enum-member-names.a24`

⚠ **The refusal is in the wrong place rather than wrong outright.** A shared member name is only a problem where a *bare* use cannot be resolved. Two enumerations that never meet such a use coexist perfectly well, and a program should be told about a name where it actually cannot be resolved — not where it was declared.

Scope of the fix. Two parts, and the first is a deletion. `VisitEnumStmt` stops refusing a member name already bound by another enumeration, and instead records that the bare name now denotes more than one member. Resolving a bare name that carries more than one binding is then the refusal, with `'A'` is ambiguous: `First` or `Second`. The qualified path already works and needs no change.

DEF-23 — Two imported modules may not export one name. (*violates [MOD-008], [MOD-013]*)

Importing two modules that both export `Shared` is refused with `'Shared'` is already defined; mark it private in one of the modules. — advice to edit a module because of what some other module, possibly written by someone else, happens to export. The qualified forms `Alpha.Shared()` and `Beta.Shared()`, which are unambiguous, never get a chance to help.

Reproduce: `defects/DEF-23-modules-may-not-share-exported-names.a24`

⚠ **The refusal is in the wrong place rather than wrong outright,** exactly as DEF-22 is for enumerations. A shared exported name is only a problem where a *bare* use cannot be resolved.

Scope of the fix. `VisitModuleStmt` stops refusing a name already exported by another imported module and instead records that the bare name now denotes more than one. Resolving a bare name carrying more than one binding is then the refusal, with `'Shared'` is ambiguous: `Alpha` or `Beta`. The qualified path already works.

DEF-24 — A cycle through the root file does not work. (*violates [MOD-014]*)

A module importing the file being run leaves the root's own imported names undefined: the root's body prints `root body ran`, and the next line is `Undefined variable 'ModName'`. A cycle between two ordinary modules works [MOD-012], so this is the root specifically.

Reproduce: `defects/DEF-24-a-cycle-through-the-root.a24`

⚠ The root is never entered in the loader's map of loaded files, so a module importing it back parses it a **second** time and the two copies do not share their names.

⚠ **It degenerates badly with a relative path.** Where the importing module names the root as `'../Root'`, the search directory accumulates `../modules` dozens of times before failing, so the diagnostic is a line of repeated path segments. A `...` path to a non-root module resolves correctly, so this is the cycle showing through rather than a fault in path resolution.

Scope of the fix. Give the root a module identity, so a `uses` pointing back at it resolves to the copy already loaded — what [MOD-003] does for every other file. ⚠ More than a guard: an import-only node carries no statements and the importer genuinely needs the root's exports, so the root must be registered with its environment **before its own body runs**.

⚠️ Fixing this also removes C-1, the only known case of a valid program having no compiled form.

DEF-25 — Length of a collection returns the length of its rendering. (violates [RT-003])

`Length([10, 20, 30])` is 12 rather than being refused: the built-in stringifies whatever it is given and measures that. `LengthNative` is literally `Exit Length(Str(Arguments[0]))`.

Reproduce: `defects/DEF-25-length-of-a-collection.a24`

⚠️ **A plausible number rather than an error**, which is the worse failure. The two are never equal — a List of n one-digit numbers renders as $3n$ characters against a count of n — so every such call is wrong by a factor of three and none of them says so.

Scope of the fix. `LengthNative` refuses a collection instead of stringifying it. ⚠️ Verified safe: every `Length(...)` call in `compiler/*.a24` is on text.

DEF-26 — A String has no Length property. (violates [RT-017])

`'abc'.Length` is Only instances have properties. A String is iterable and subscriptable but does not answer for its own length, while every collection does.

Reproduce: `defects/DEF-26-a-string-has-no-length-property.a24`

⚠️ **The compiled back end already answers 3** (C-9). With [COL-006] this is one of only two rules where the C back end is right and the interpreter is the one to change — everything else in Annex C runs the other way.

Scope of the fix. The property path answers `Length` for a String from its character count, as `alg_property` already does in the C runtime.

DEF-27 — Val and Max cannot be used together. (violates [RT-009], [RT-010])

`Val('42')` is `42.0` rather than `42`, and `Max` refuses anything but Integers, so `Max(Val(A), Val(B))` is `Max` expects Integers. for **every** input.

Reproduce: `defects/DEF-27-val-and-max-do-not-meet.a24`

⚠️ **One defect, not two.** Each built-in is defensible alone; the fault is that the output of one cannot be the input of the other, and no program can work around it without leaving both.

Scope of the fix. `ValNative` reads the text the way [LEX-015] and [LEX-020] do and answers the matching type; `MaxNative` accepts two numbers and promotes. ⚠️ It should land with DEF-10 — once widening works, `Max` promoting is the same rule the operators use rather than a special case in one built-in.

DEF-28 — A file that cannot be read exits 0. (violates [INI-005])

```
$ bootstrap/alg /no/such/file.a24 ; echo $?
alg: cannot open /no/such/file.a24
0
```

```
$ bootstrap/alg compiler ; echo $?  
0
```

The driver reports a failure and reports success at the same time, so no caller can tell a program that ran from one that was never found. Naming a directory does not even print.

⚠ **No case in** `defects/` **reproduces this, and none can.** The fault is in the driver, and every case in the corpus is run by handing `alg` a file that exists. A case that ran and exited 0 would record exactly what a correct implementation produces — a defect test that passes for the wrong reason, which is the failure this corpus has already been caught by twice (the CRLF probe got normalized, and the first draft of DEF-16). The shell commands above are the reproduction.

Scope of the fix. The three `alg: cannot open sites in compiler/Main.a24` raise rather than returning, so the failure reaches the same exit path every other failure uses [INI-006]. A directory should say so rather than printing nothing.

DEF-29 — A type error says nothing. (*violates [ERR-006]*)

```
Uncaught: Type mismatch!
```

is the whole diagnostic, for every mismatch the checker finds — no file, no line, no token, neither of the types involved, and the same five words each time.

Reproduce: `defects/DEF-29-a-type-error-says-nothing.a24`

⚠ **The machinery exists and is used two phases earlier.** A parse error prints the file, the line, the source text and a caret under the offending token [ERR-005] — compare `conformance/0100` against the recording above for the same shape of program. `Console.Error` produces it, and the checker does not call it.

⚠ **It compounds with [ERR-002],** which stops the program before any statement runs, so there is no output to orient by either. And it will compound with DEF-19: once parameter types are enforced, mismatches surface in *more* places than they do now, each reported by five words that say nothing.

Scope of the fix. Five sites in `compiler/TypeChecker.a24` raise this string. Each has the offending expression in hand, so each can report through `Console.Error` with the token and both type names — `Expected Integer, found String`. The information is discarded rather than absent.

DEF-30 — Every assertion message begins with a comparison that was not made. (*violates [TST-012]*)

All three assertions produce a message beginning `Assertion 'left = right' failed.` — a stem naming operands that no message ever fills in, reading like a template nobody finished.

For `AssertTrue` it is worse than untidy: it describes an equality test that did not happen and says nothing at all about the value that was false.

```
AssertTrue (False) → Assertion 'left = right' failed.
```

Reproduce: defects/DEF-30-assertion-messages.a24

⚠ **These three messages are read more often than any other output this language produces**, because a failing test is the one moment a programmer is looking at output rather than writing code.

⚠ The type-naming form is **correct and must be kept**: a `Char` and a `String` both render as `a`, so a message quoting only the rendered values would read `Expected 'a' but got 'a'`. Both processors already produce it, and [TST-012] had omitted it from its table entirely.

Scope of the fix. Three sites in `compiler/Interpreter.a24` and two in `bootstrap/algol.c`. ⚠ It changes the **report**, which [TST-008] specifies and which both processors must reproduce byte for byte — so the interpreter and the C runtime must change together, and `conformance/0105` records the current text. No test in `compiler/*.a24` asserts on these strings, so the suite itself is unaffected.

DEF-31 — A test cannot have a one-character name. (*violates [TST-001]*)

```
test 'X';
begin
    AssertTrue (True);
end
```

is not recognised as a test declaration. `'X'` is a `Char` rather than a `String` [LEX-023], and only a `String` is accepted, so this parses as the identifier `test` followed by a `Char` and fails with `Expect ';' after expression.` — a diagnostic naming neither tests nor names.

Reproduce: defects/DEF-31-a-one-character-test-name.a24

⚠ The name of a test is **text**, and the `Char/String` distinction belongs to values rather than to a declaration naming itself.

Scope of the fix. The lookahead in `compiler/Parser.a24` 's `Declaration` accepts either quoted form. ⚠ It must accept both rather than switching, or every existing test name breaks.

Annex G — implementation notes (*non-normative*)

Guidance for implementers. Nothing here is a rule; an implementation may reach these ends by other means.

G.1 Reporting a scan error

[ERR-004] specifies the shape of a scan error and that it prevents execution. It deliberately says nothing about the mechanism, because this implementation's mechanism has a hazard worth describing rather than

mandating.

`compiler/Scanner.a24` **records** rather than raises: it sets `HadError` and keeps the message in `LastError`, and a driver must ask. Two consequences follow:

- **A driver that does not ask carries on** over a damaged token stream. `compiler/Main.a24` calls `CheckScanned` **twice** — once after scanning for the right message, and once after parsing so that modules loaded by `uses` are covered — and both calls are load-bearing.
-  `HadError` **and** `LastError` **are module-level, not per instance**. Two scanners in one process share them, so an error from one is visible to the other and a later clean scan does not clear an earlier failure. Nothing in the language reaches this — a program cannot construct a `Scanner` — but any tool that scans more than one source in a process can.

An implementation that raises instead has neither problem and conforms equally.

G.2 Mangling identifiers into C

The C back end must map an Algol-24 identifier — which may hold `_`, any Unicode letter, and the marks `?` and `!` [SRC-005] — onto a C identifier, which may not.

Because identifiers are case-insensitive [SRC-011], **case carries no information**, and lowercasing the name is lossless. That frees the entire uppercase range to act as escape markers:

Source	Emitted
any letter or digit	lowercased
<code>?</code>	<code>Q</code>
<code>!</code>	<code>E</code>
<code>_</code>	<code>V</code>
any other character	<code>U</code> followed by six hexadecimal digits

`_` remains unused by the mapping and is therefore available as a separator wherever two escapes would otherwise run together ambiguously.

The existing per-kind prefixes — `v_` a variable, `f_` a function, `fn_` its closure, `k_` a class, `e_` an enum, `c_` a constant, `m_` a method — continue to keep the emitter's names clear of C's.

 **This scheme is injective, and the one it replaces is not.** Today `?` becomes `_q`, so `Ready?` and `Ready_q` emit one symbol between them and `cc` refuses the result — a collision `CEmitter.a24` documents against itself. Under the scheme above they become `readyQ` and `readyVq`, which differ.

 The two decisions depend on each other. Lowercasing is only lossless because identifiers are case-insensitive, and the uppercase escape space only exists because of the lowercasing. Neither works alone.

Annex H — planned for later generations (*non-normative*)

Changes intended for the language that are **not defects**. The implementation is right about these; the specification describes the language as it now is, and each of these will change a rule when it arrives.

⚠ The distinction matters to the corpus. A defect gets a reverse conformance test that passes while the wrong behaviour persists. A planned change gets an ordinary conformance test or refusal pinning the **current** rule, which turns red when the generation lands — deliberately, because that is the moment the rule changes and the test should change with it.

H-1 — Other bases and digit separators. (*will change [LEX-016]*)

Hexadecimal, octal and binary integer literals, and a separator within a run of digits. Today decimal is the only base and there is no separator; `0x10` scans as `0` followed by the identifier `x10`, and the adjacent expressions do not parse. Pinned by `refusals/0006-hex-is-not-a-literal.a24`.

H-2 — Exponent notation. (*will change [LEX-022]*)

A double literal with an exponent. Today `1e5` scans as `1` followed by the identifier `e5`. Pinned by `refusals/0007-exponent-is-not-a-literal.a24`.

⚠ This one closes a genuine asymmetry rather than only adding a convenience: `Str` **prints** a large `Double` in exponent form — `1.0E300` — and the language cannot read back what it wrote. Nothing is unreachable today, because `Val` parses the exponent form, but the round trip goes through a built-in rather than through the source.

H-3 — Element types checked on insertion. (*will change [VAR-016]*)

An element type constrains what may be put into a collection, so `L.Add('text')` on a `List of Integer` is refused. Today it is a source of types for reads only and insertion is unchecked. Pinned by `conformance/0021-element-types-flow-to-reads.a24`, whose `Add` line is the part that will change.

⚠ Larger than it looks, and the reason it is not being done now: every route into a collection has to be covered — `Add`, `Put`, `Push`, subscript assignment, and the collection literals — or the check becomes a fence with a gate in it, which is worse than no fence because it invites the declared type to be trusted.

H-4 — A subscript operator a class may declare. (*will change [TYP-010]*)

`B[0]` on a class instance is `Subscript` target should be an ordinal. today, whatever methods the class declares. Pinned by `conformance/0031-instance-is-not-subscriptable.a24`.

H-5 — An iteration protocol a class may implement. (*will change [TYP-011]*)

for var X in B over a class instance is Can only iterate a collection or a String. today.
Pinned by conformance/0032-instance-is-not-iterable.a24 .

H-6 — A computed property. *(will change [TYP-012])*

A method read without parentheses yields the method. There is no getter, so a class cannot expose a computed Length the way every collection does. Pinned by conformance/0033-no-computed-property.a24 .

⚠ **H-4, H-5 and H-6 are one piece of work, not three.** They are exactly what Annex E identifies as pinning the collections to being native: a Stack written in Algol-24 cannot be subscripted, cannot be iterated, and cannot answer Length without parentheses. Any one of them alone leaves a user-written collection visibly second-class, so the generation that brings them should bring all three — and Annex E's estimate of what could then move out of the runtime depends on it.

H-7 — Ordering for Strings. *(will change [VAL-014])*

'ab' < 'cd' is the runtime error Operands must be numbers. today, so a program needing to order text compares it character by character — compiler/CEmitter.a24 writes TextLess for exactly this. Pinned by conformance/0038-strings-are-not-ordered.a24 .

H-8 — Operator overloading. *(will change [VAL-011], and more)*

An operator a program may define for its own type. Today comparison of a class instance is by identity [VAL-011] with no way to say otherwise, and a user-written collection cannot be compared, ordered or combined the way a built-in one can.

⚠ **This is the umbrella over much of Annex H, not another item in it.** H-4's subscript operator and H-6's computed property are operator overloading in two particular spellings, and H-7 is the built-in case of an ordering that overloading would let a program supply for itself. Several rules in chapters 6 and 7 are shaped by the language having no answer here, and they will want revisiting together rather than one at a time.

H-9 — The collections as a unit written in Algol-24. *(will retire most of chapter 14)*

Everything in chapter 14 that is not genuinely pinned moves out of the core and into a unit written in Algol-24. Annex E is the survey of what is pinned and by what; in short:

Array	stays. Fixed-size, constant-time, holding arbitrary values — nothing in the language can express it, and it is the primitive the others are built on.
Stack	movable today. Its whole surface is method calls and no literal claims its name. The best first candidate.
Set	movable today with a linear scan; hashable later using Ord .
List	pinned by [...] , and not worth moving until H-4 and H-5 arrive.

Map	pinned by <code>[:]</code> and <code>[k : v]</code> , and by wanting <code>M[K]</code> .

⚠ **H-4, H-5 and H-6 are prerequisites for the last two**, not merely desirable alongside them. Without a subscript operator, an iteration protocol and a computed property, a `List` written in Algol-24 reads `L.Get(I)` and `L.Length()` and needs an index loop — strictly worse to use than the built-in it replaces, which is not a trade worth making.

⚠ **[COL-007] is the constraint that outlives the move.** Insertion order is specified for every collection, `Set` and `Map` included, so a unit is a third implementation bound by it exactly as the two processors are.

⚠ **The point is Annex C.** Everything moved out of the runtime is one thing the two processors can no longer disagree about, and that annex is now fifteen entries long. `spec/probes/TYP-012-stack-and-set-in-algol24.a24` already holds a working `Stack` and `Set` written in the language, so the path is not speculative.

⚠ When a collection moves, its rules leave this specification and its conformance cases become **unit tests of the unit**. A rule retired because its subject became a library has not been falsified — the distinction matters to anyone reading `conformance/` later and wondering where the cases went.